CONF42 · LLMs 2026 · JUNE 18

How vLLM-Omni Unifies Multimodal Inference

One engine. Two brains. **Any-to-any inference.**

See it. Hear it. Speak it. Generate it

Kosseila “CloudDude” Hd

CloudThrill · cloudthrill.ca · X: @clouddude_

Intro generated with AI video ·

Credit @anselmovitor

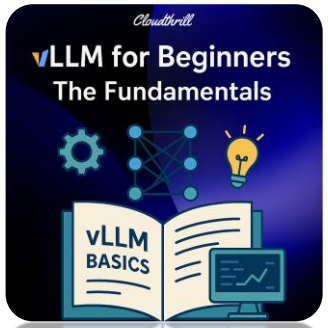
CloudThrill
expertise worth sharing...

$ WHOAMI

Kosseila Cloud Dude

Founder @ CloudThrill · 18+ years infra → AI infrastructure lead · vLLM contributor · Nebius Fellow

...but forget the bio. **Here's the part that's actually useful to you →**



vLLM for Beginners

Parts 1 · 2 · 3

fundamentals, features, deployment — Deep dive

KV-Cache Explained

the memory trick

why inference is memory-bound, and how vLLM pages it

Diffusion, Explained

noise → pixels

how image & video models actually generate — the core of Omni

All available at **cloudthrill.ca/blog**

...and more freebies drop later in this deck. Keep watching.

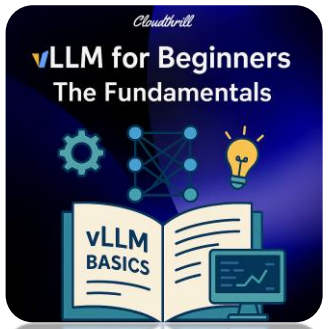
$ WHOAMI

Kosseila Cloud Dude

Founder @ CloudThrill · 18+ years infra → AI infrastructure lead · vLLM contributor · Nebiu

CloudThrill
expertise worth sharing...

...but forget the bio. **Here's the part that's actually useful to you →**



vLLM for Beginners

Parts 1 · 2 · 3

fundamentals, features, deployment — Deep dive

KV-Cache Explained

the memory trick

why inference is memory-bound, and how vLLM pages it

Diffusion, Explained

noise → pixels

how image & video models actually generate — the core of Omni

All available at **cloudthrill.ca/blog**

...and more freebies drop later in this deck. Keep watching.

THE PLAN · 45 MIN

Agenda

01 **The Multimodal Mess** Why text, Image, video, Audio serving (AR, DiT, hybrids)— breaks your stack

02 **One Engine, Two Brains** When autoregressive meets diffusion — the architecture, Stages, OmniRouter, OmniConnector

03 **Serving & Deployment (orchestration)**— a request's journey

04 **Omni Acceleration features** TeaCache, Cache-DiT, SP, TP, HSDP, Ring, VAE-patch, CFG, quant

05 **Demo(Image, Video, Cosmos) + bench** Image · video · speech on Nebius H100 Node

06 **v0.22 & Beyond** world models, robots, and the road ahead

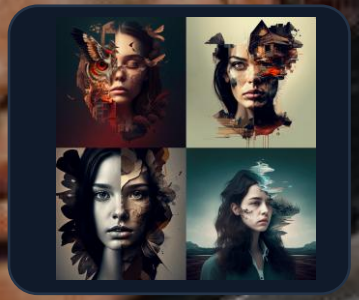
I · THE MULTIMODAL MESS

You've probably used one of these this year.

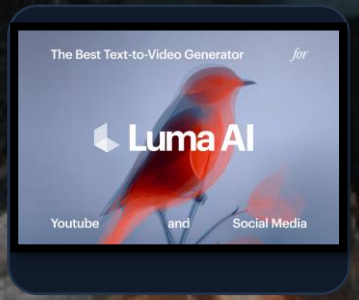
Google Veo3

Eleven labs

Kling



Midjourney



Luma

Seedance

One thing in common: they all run on diffusion models.

I · THE MULTIMODAL MESS

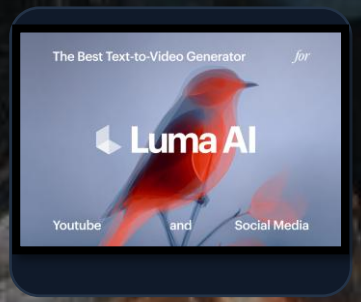
You've probably used one of these this year.

Google Veo3

Eleven labs

Kling

Midjourney



Luma

Seedance

THE CATCH:

Serving these models isn't like serving an LLM.

I · THE MULTIMODAL MESS

❑ Modern AI is a mess of text, image, video, and audio.

Text = autoregressive

token by token, KV-cache, memory-bound

one token at a time

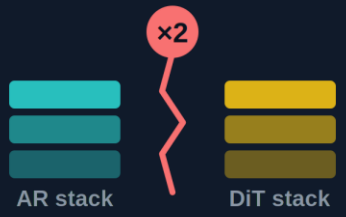
Images/video = diffusion

denoising steps, compute-bound, totally different physics

noise to image

So teams ran two stacks

two engines, two schedulers, two ops burdens — to serve one product



THE CATCH:

Serving these models isn’t like serving an LLM.

❑ Most inference engines were never built for multimodality.

I · THE MULTIMODAL MESS

❑ Modern AI is a mess of text, image, video, and audio.

Text = autoregressive

token by token, KV-cache, memory-bound

one token at a time

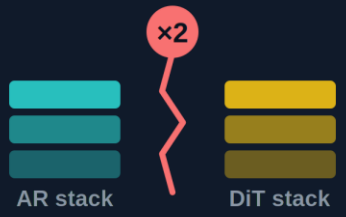
Images/video = diffusion

denoising steps, compute-bound, totally different physics

noise to image

So teams ran two stacks

two engines, two schedulers, two ops burdens — to serve one product

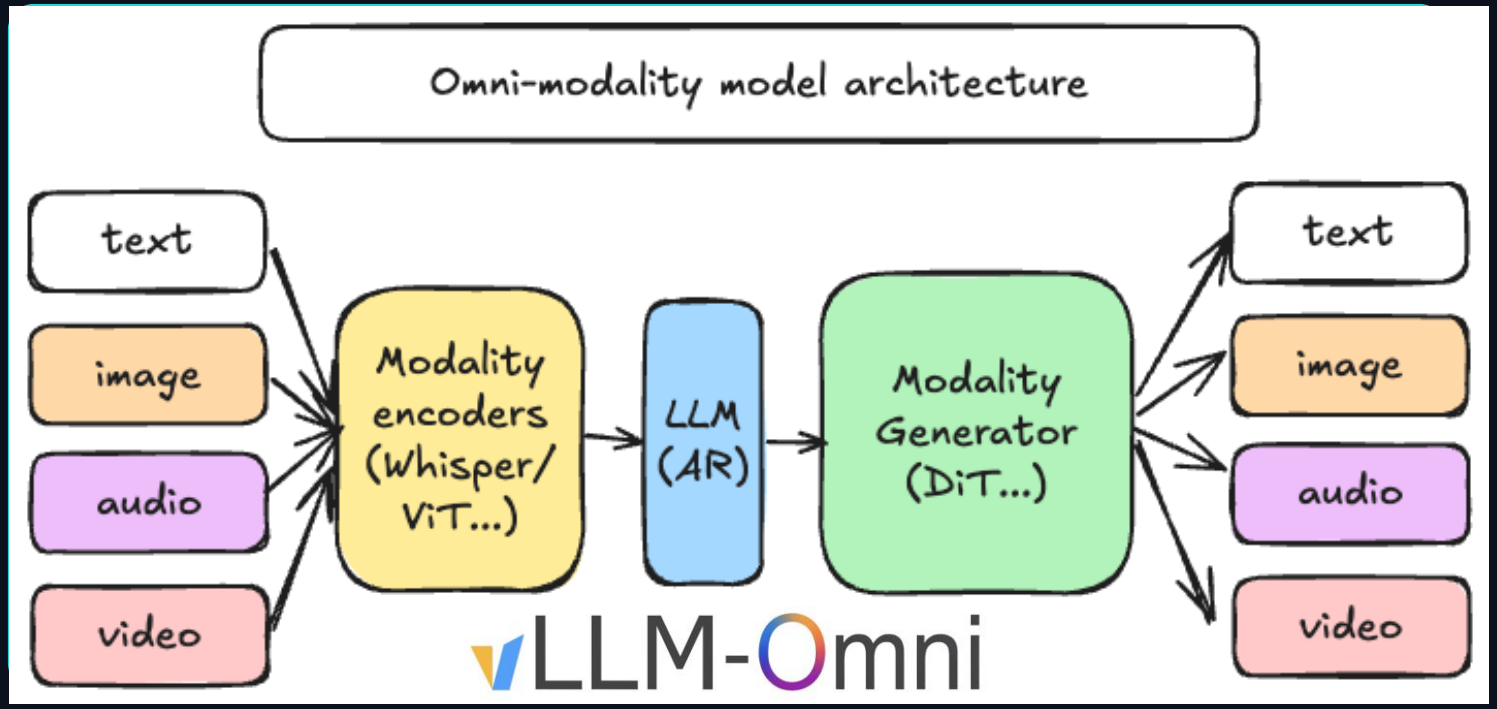


THE CATCH:

What if one engine could serve both? That's vLLM-Omni.

Omni-Modality models

Omni-modality: Text, image, video, and audio data processing



Non-autoregressive Architectures: extend the AR support of vLLM to Diffusion Transformers (DiT) and other parallel generation models

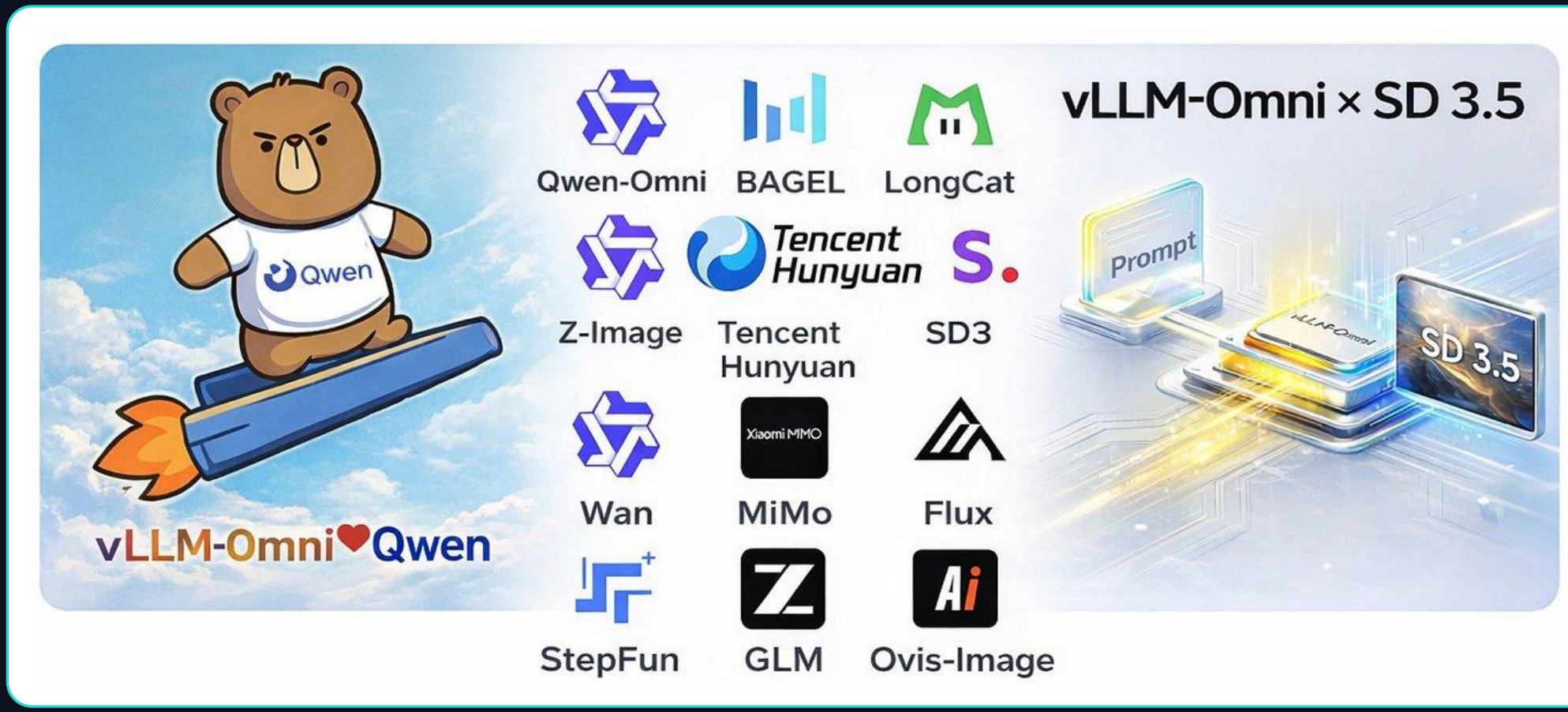
Heterogeneous outputs: from traditional text generation to multimodal outputs

I · THE MULTIMODAL MESS

40+ architectures. One engine.

BUILT IN THE OPEN

5.1k stars in GitHub



source: vLLM-Omni team · vLLM Office Hours #46 / project docs · verify current model count before recording

I · THE MULTIMODAL MESS

40+ architectures. One engine.

BUILT IN THE OPEN

5.1k stars in GitHub

source: vLLM-Omni team · vLLM Office Hours #46 / project docs · verify current model count before recording

II · ONE ENGINE, TWO BRAINS

Two brains, opposite physics (AR vs DiT)

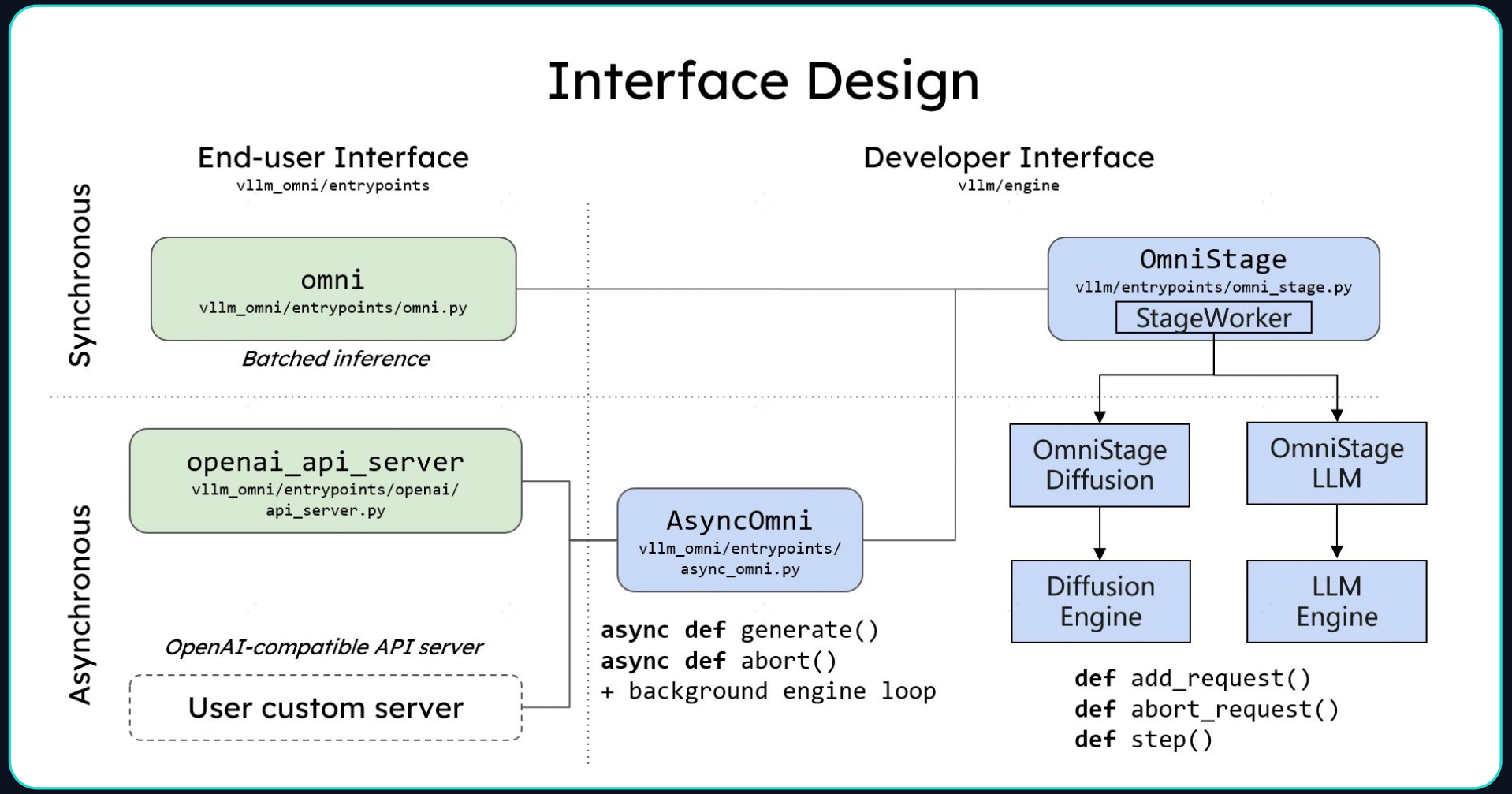
	AR (the talker)	DiT (the dreamer)
Generates	text, token by token (Thinker/Talker)	pixels & waveforms, denoising steps (Wan , FLUX, ..)
Driven by	LLMs (KV-cache)	iterative refinement (every step = full forward Dit pass)
Bottleneck	prefill: compute (Kvcache) · decode: memory	Compute bound (High-throughput generation)
Sequence length	varies per request	fixed by output size
Attention mask	causal	full
Parallelism	TP · DP · EP · PP · CP · SP	TP · EP · USP · CFG

Opposite bottlenecks → historically, separate serving stacks. vLLM-Omni runs both under one runtime.

adapted from: vLLM-Omni team · vLLM Office Hours #46

II · ONE ENGINE, TWO BRAINS

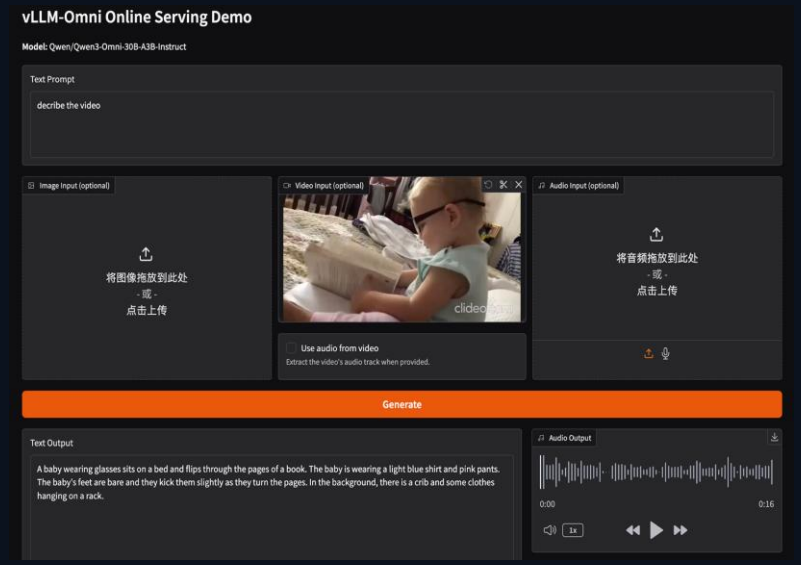
One runtime, many doors



source: vLLM-Omni team · vLLM Office Hours #46 / project docs

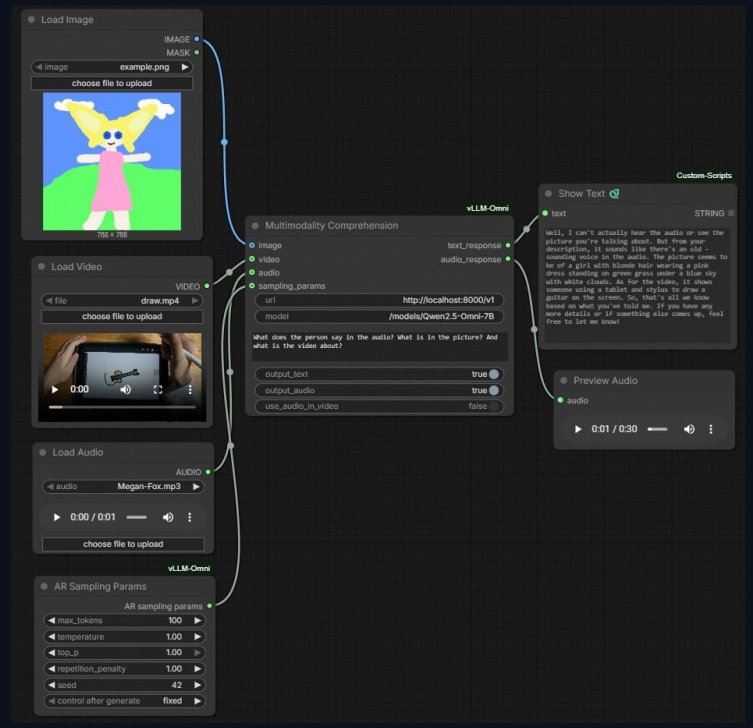
vLLM-Omni User Interface (GUI)

A Gradio demo for Qwen3-Omni online serving



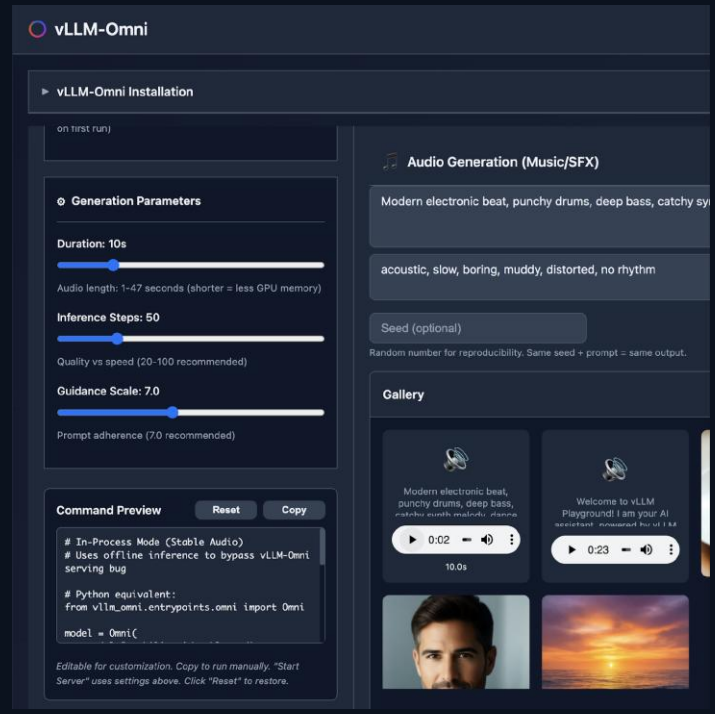
https://docs.vllm.ai/projects/vllm-omni/en/latest/user_guide/examples/online_serving/qwen3_omni/

ComfyUI Integration for low-code development



https://docs.vllm.ai/projects/vllm-omni/en/latest/features/comfyui/

vllm-playground: A web interface for interacting with vLLM servers



https://github.com/micytao/vllm-playground

vLLM-Omni User Interface (GUI)

Server

```
$ vllm serve Qwen/Qwen-Image-2512 --omni --port 8091
```

Client

```
$ curl -X POST http://localhost:8091/v1/images/generations \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "a dragon laying over the spine of the Green Mountains of Vermont",
    "size": "1024x1024",
    "seed": 42
  }' | jq -r '.data[0].b64_json' | base64 -d > dragon.png
```

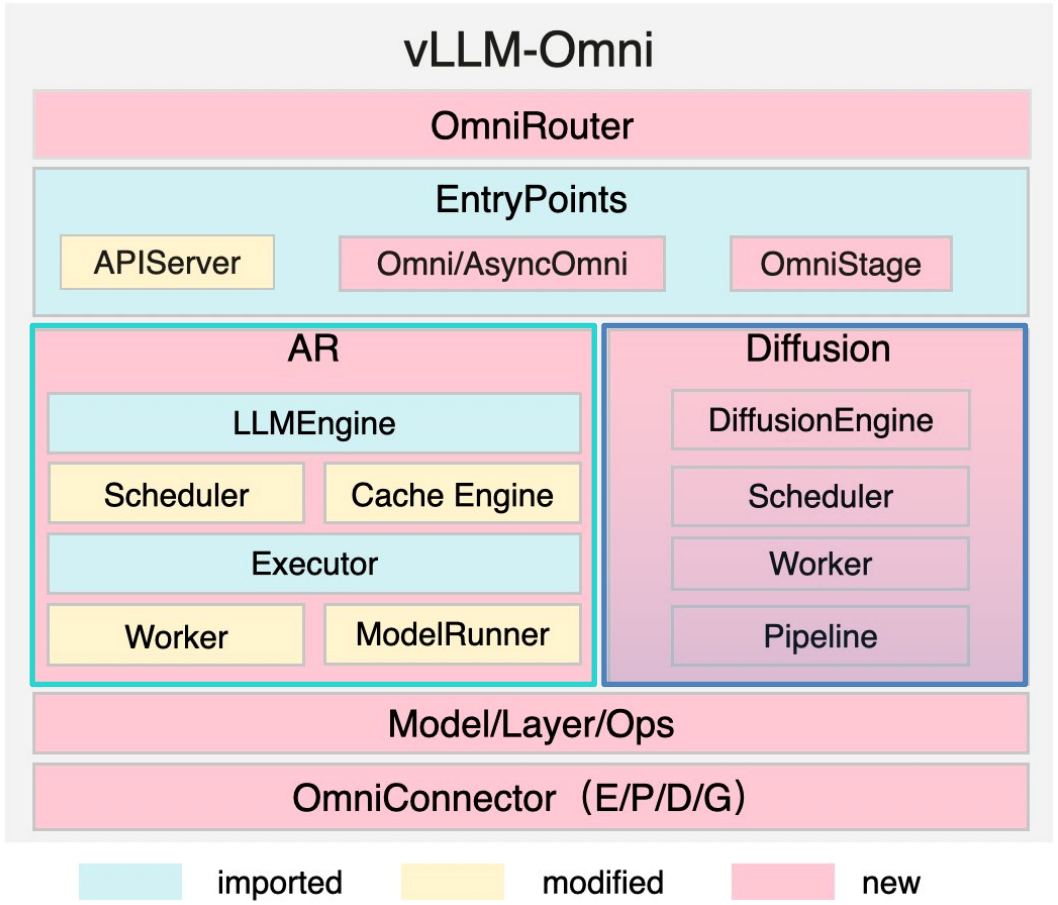
https://docs.vllm.ai/projects/vllm-omni/en/latest/user_guide/examples/online_serving/qwen3_omni/

https://docs.vllm.ai/projects/vllm-omni/en/latest/features/comfyui/

https://github.com/micytao/vllm-playground

II · ONE ENGINE, TWO BRAINS

One runtime, two engines inside



OmniRouter INTELLIGENCE
dispatches each request to the optimal unit — by modality and load

EntryPoints API LAYER
offline/online APIs (APIServer, Omni/AsyncOmni) + the OmniStage abstraction

AR AUTOREGRESSIVE
inherited from vLLM — KV-cache management, PagedAttention — adapted for omni

Diffusion GENERATIVE
natively built, tuned by acceleration components for image & video throughput

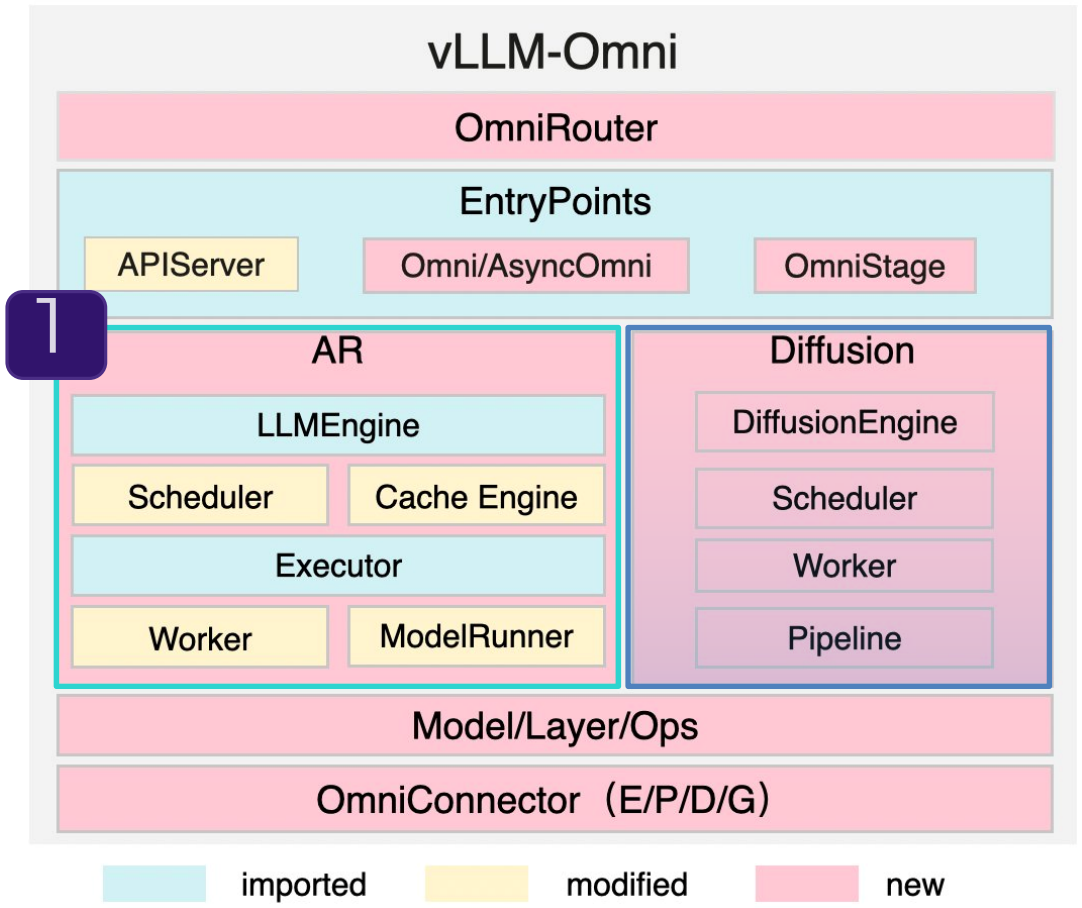
Model/Layer/Ops KERNELS
parallelism · quantization · attention

OmniConnector DISAGGREGATION
full E/P/D/G split — Encoding, Processing, Decoding, Generation across stages

source: vLLM-Omni team · vLLM Office Hours #46 / project docs

II · ONE ENGINE, TWO BRAINS

One runtime, two engines inside



OmniRouter INTELLIGENCE
dispatches each request to the optimal unit — by modality and load

EntryPoints API LAYER
offline/online APIs (APIServer, Omni/AsyncOmni) + the OmniStage abstraction

AR AUTOREGRESSIVE
inherited from vLLM — KV-cache management, PagedAttention — adapted for omni

Diffusion GENERATIVE
natively built, tuned by acceleration components for image & video throughput

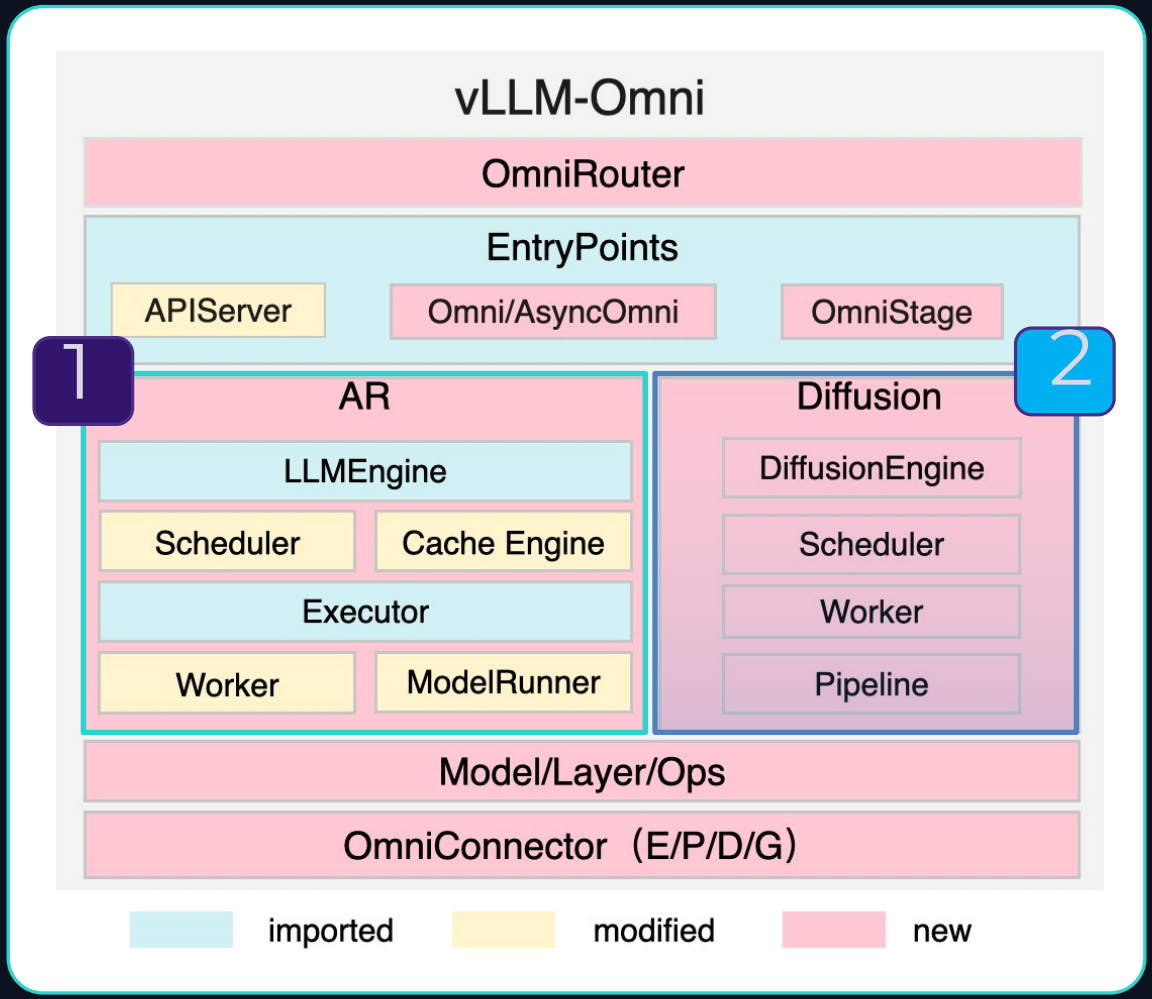
Model/Layer/Ops KERNELS
parallelism · quantization · attention

OmniConnector DISAGGREGATION
full E/P/D/G split — Encoding, Processing, Decoding, Generation across stages

source: vLLM-Omni team · vLLM Office Hours #46 / project docs

II · ONE ENGINE, TWO BRAINS

One runtime, two engines inside



OmniRouter INTELLIGENCE
dispatches each request to the optimal unit — by modality and load

EntryPoints API LAYER
offline/online APIs (APIServer, Omni/AsyncOmni) + the OmniStage abstraction

AR AUTOREGRESSIVE
inherited from vLLM — KV-cache management, PagedAttention — adapted for omni

Diffusion GENERATIVE
natively built, tuned by acceleration components for image & video throughput

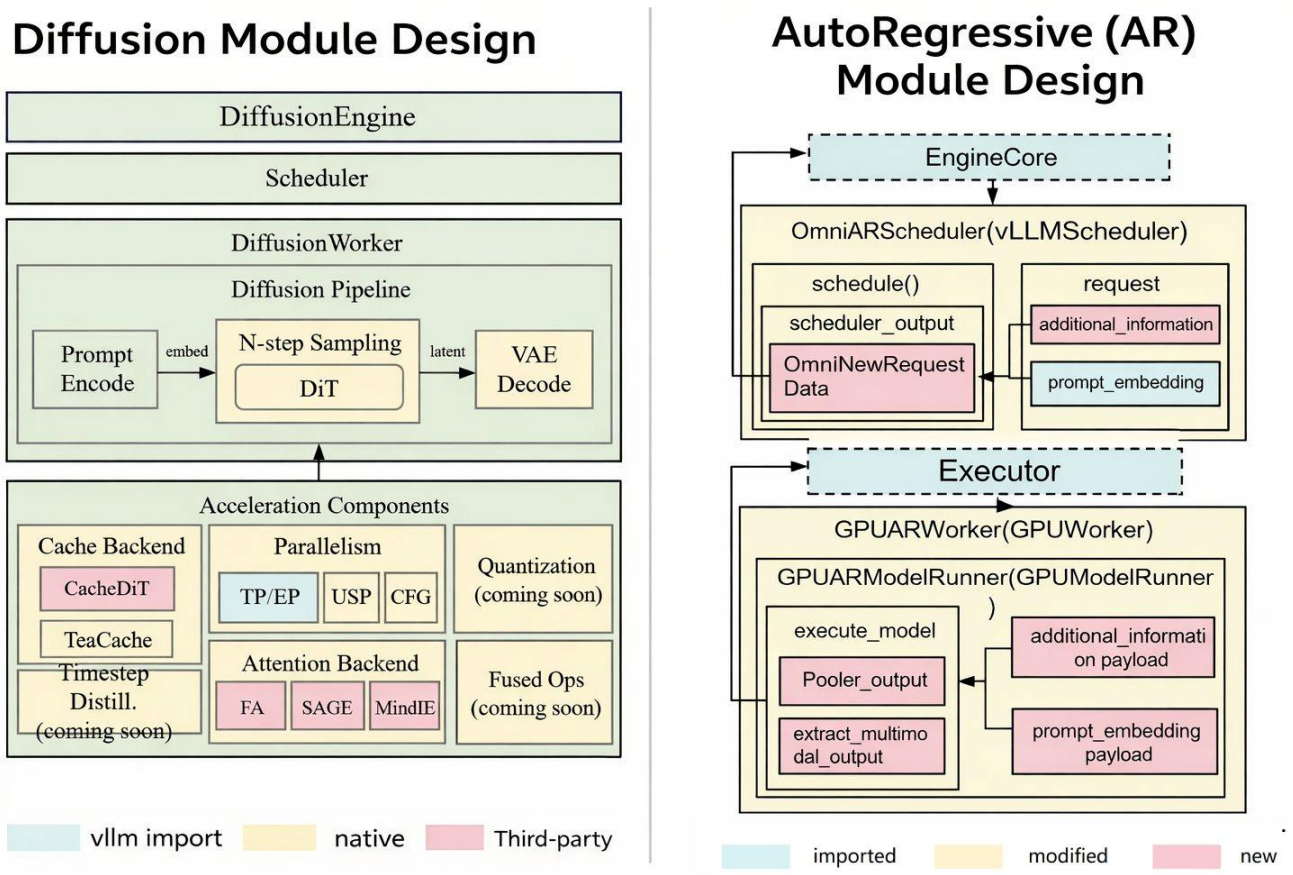
Model/Layer/Ops KERNELS
parallelism · quantization · attention

OmniConnector DISAGGREGATION
full E/P/D/G split — Encoding, Processing, Decoding, Generation across stages

source: vLLM-Omni team · vLLM Office Hours #46 / project docs

II · ONE ENGINE, TWO BRAINS

Inside the two brains — module design



Diffusion Core

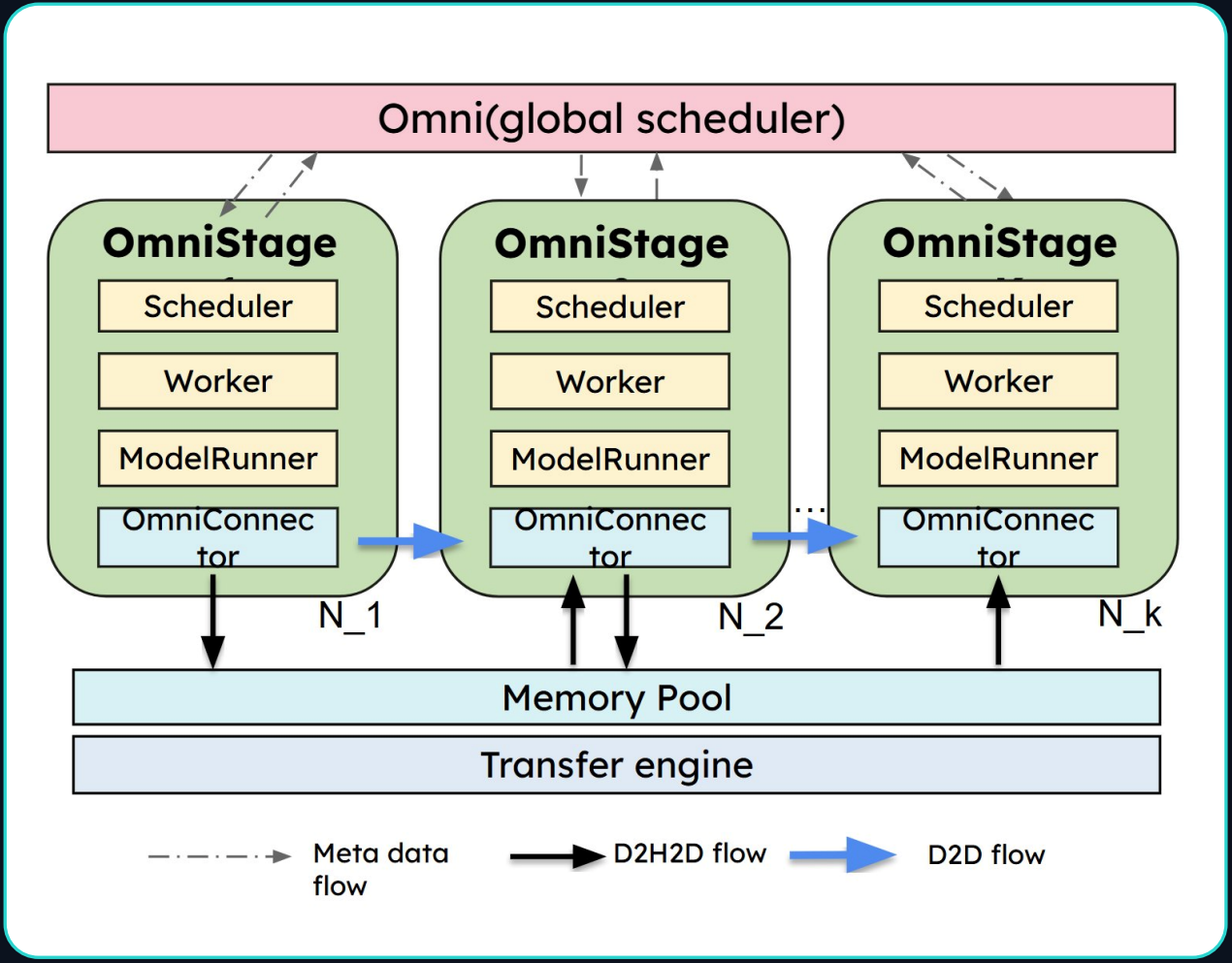
- Natively optimized by configuring the acceleration layer
- Aligned with the AR arch with model_runner abstraction
- Encoders are disaggregated to run on vllm engine for higher throughput in large-scale deployment
- Support step-wise scheduler with continuous batch
- Connect with AR module to speed up AR+DIT model inference with OmniConnector

Acceleration features

- Cache backend, Parallelism, Quantization:
- CPU Offload: module-wise/layer-wise
- Extensions: LoRA

source: vLLM-Omni team · vLLM Office Hours #46 / project docs

Stages talk through OmniConnector



source: vLLM-Omni team · vLLM Office Hours #46 / project docs

Control vs data plane
metadata travels light; heavy tensors ride the fast path

D2D when possible
GPU-to-GPU direct — no host round-trip

D2H2D when needed
memory pool + transfer engine (SHM, Mooncake) across nodes

Scale per stage
each OmniStage replicates independently — N_1, N_2 … N_k

*D2D: Device to Device
*D2H2D: Device to Host to Device

II · ONE ENGINE, TWO BRAINS

Pick your transport — intra-node vs across the cluster

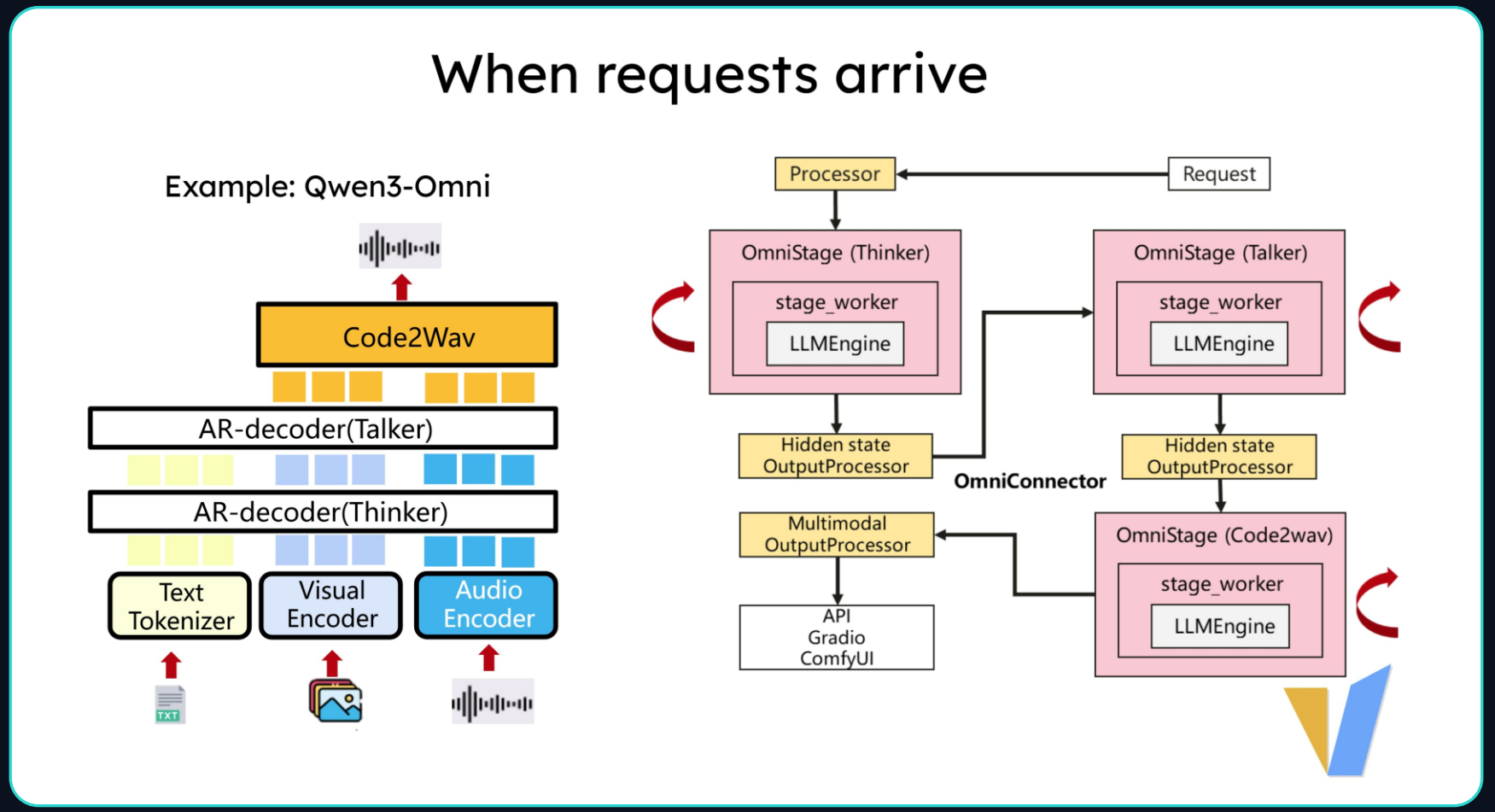
CONNECTOR	SCOPE	TRANSPORT	USE CASE
`SharedMemoryConnector`	SINGLE NODE	`/dev/shm`	local stages, lowest latency — multiprocessing.shared_memory
`MooncakeStoreConnector`	CROSS NODE	`RDMA / TCP`	cross-node, or high-bandwidth intra-node — via Mooncake store
`MooncakeTransferEngineConnector`	CROSS NODE	`RDMA`	optimized transfer engine for large clusters

Same OmniConnector seam, swappable transport: Shared memory on **one box**, Mooncake when you **cross the network**.

source: vllm-omni docs · connector types

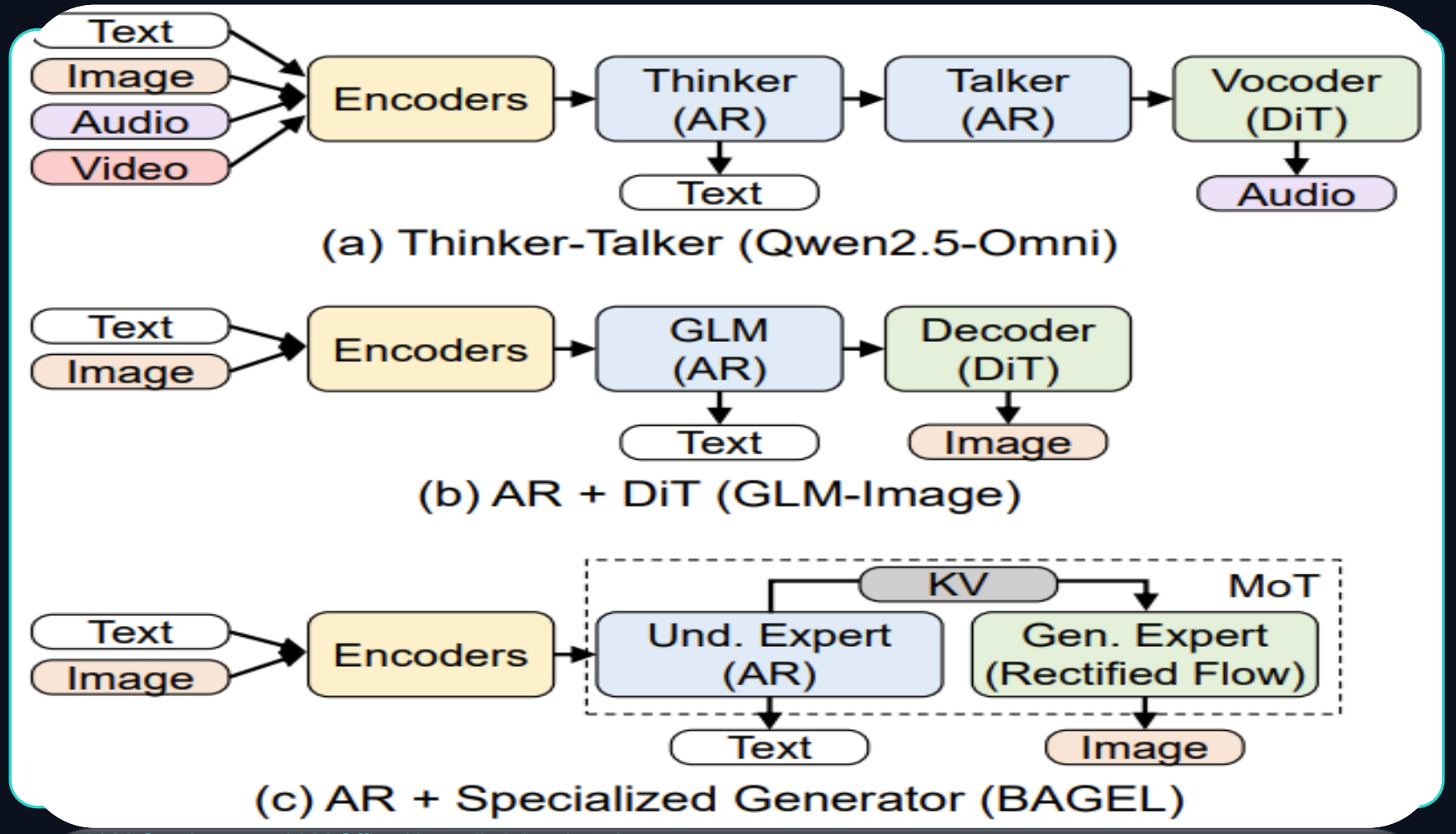
II · ONE ENGINE, TWO BRAINS

A request's journey — Thinker → Talker → Code2wav (TTS)



source: vLLM-Omni team · vLLM Office Hours #46 / project docs

A request's journey — Thinker → Talker → Code2wav (TTS)



source: vLLM-Omni team · vLLM Office Hours #46 / project docs

II · ONE ENGINE, TWO BRAINS

The pipeline, stage by stage (TTS)

STAGE	FUNCTION	OUTPUT / DATA FLOW
THINKER *Understanding*	**Multimodal understanding.** Ingests text, image, video or audio — produces reasoning states.	Text tokens + hidden-state embeddings (layers 0 & 24) → handed to Talker.
TALKER *Codec logic*	**Speech synthesis.** Converts Thinker's text + hidden embeddings into multi-layer RVQ codec codes.	16 layers of RVQ codes → streamed downstream to Code2Wav.
CODE2WAV *Waveform*	**Audio decoding.** Transforms RVQ codes into a high-fidelity audible waveform.	Final 24 kHz waveform tensor → returned to the client.

content: vLLM-Omni docs · Qwen-Omni stage configs

III · SERVING & DEPLOYMENT

Config: One model, three stages — declared in YAML

```
OmniStage_0
Thinker GPU 0
model_stage: thinker
worker_type: ar
gpu_mem_util: 0.9
output_type: latent
devices: "0"
final_output: text
```

→

```
OmniStage_1
Talker GPU 1
model_stage: talker
input_source: [0]
gpu_mem_util: 0.6
output_type: latent
devices: "1"
```

→

```
OmniStage_2
Code2wav GPU 1
model_stage: code2wav
input_source: [1]
gpu_mem_util: 0.1
final_output: audio
devices: "1"
```

Text in → reasoning → speech tokens → waveform out. Each stage = its own engine, its own GPU budget (.i.e: 3stages 2GPU)

Scale per stage

each OmniStage replicates independently — N_1, N_2 … N_k

III · SERVING & DEPLOYMENT

Pipeline vs Deploy Confusion — code vs placement

pipeline.py the WHAT

Model code — ships with the model integration.

- defines the stages: Thinker → Talker → Code2Wav
- what each stage computes, what it hands downstream
- fixed per model — you don't touch this to deploy

`vllm_omni/model_executor/models/`

`qwen3_omni/pipeline.py`

deploy YAML the WHERE

Placement & runtime knobs — yours to override.

- **D**evices per stage · gpu_memory_utilization · max_num_seqs
- **O**verlay pattern: base_config + override just one stage
- **S**wap hardware? edit YAML — never the model code

`vllm_omni/deploy/qwen3_omni_moe.yaml`

`+ --deploy-config your_override.yaml`

i.e Qwen3-Omni → 3 stages on 2 GPUs (Thinker GPU0, Talker + Code2Wav share GPU:1) or 3 stage - as many GPUs

Stages ≠ GPUs

A stage is a unit of work, not a unit of hardware. Squeeze 3 stages onto 2 GPUs, or fan a stage out to N replicas (`qwen3_omni_moe_multi_replicas.yaml`).

*source: vllm-omni repo · config: **deploy/qwen3_omni_moe.yaml** · test: **examples/online_serving/qwen3_omni/README.md***

III · SERVING & DEPLOYMENT

Same model, three ways to launch

```
# 1 · one box — hand it your placement file
$ vllm serve Qwen/Qwen3-Omni-30B-A3B-Instruct --omni --port 8091 --deploy-config /path/to/your_deploy_config.yaml
```

```
# 2 · One process per stage — stage 0 owns the API server
$ CUDA_VISIBLE_DEVICES=0 vllm serve ... --omni --port 8091 \
      --stage-id 0 --omni-master-address 127.0.0.1 \
      --omni-master-port 26000
$ CUDA_VISIBLE_DEVICES=1 vllm serve ... --omni --headless \
      --stage-id 1 --omni-master-address 127.0.0.1 ...
$ CUDA_VISIBLE_DEVICES=1 vllm serve ... --omni --headless \
      --stage-id 2 --omni-master-address 127.0.0.1 ...
# stages 1 & 2 dial the master — no API server of their own
```

```
# 3 · inline per-stage overrides
$ vllm serve ... --omni \
  --stage-overrides '{
   "0": {"gpu_mem...": 0.1},
   "1": {"gpu_mem...": 0.5},
   "2": {"gpu_mem...": 0.2}
  }'
# tune one stage, YAML defaults for rest
```

Placement is an ops decision, not a code change : pick the launch shape that fits your hardware.

source: vllm-omni docs · : examples/online_serving/qwen3_omni/README.md

III · SERVING & DEPLOYMENT

The repo, mapped — where things live

DIRECTORY	ROLE	WHAT LIVES THERE
`vllm_omni/deploy/`	NEW SCHEMA	user-editable runtime YAMLs, auto-loaded — gpu mem, devices, connectors (migrated: Qwen3-Omni, TTS…)
`model_executor/stage_configs/`	LEGACY	old stage_args format, load via --stage-configs-path — Wan2.2's fp8 config still lives here
`model_executor/models/`	AR / OMNI	Thinker/Talker/Code2wav implementations — OmniARScheduler, continuous batching
`diffusion/models/`	DIT	pipeline_*.py + *_transformer.py per model — DiffusionEngine, step-wise sampling
`recipes/  (external repo)`	DOCS	per-model setup guides & tuning recipes — community-editable, outside the codebase
`examples/online_serving/`	RUNNABLE	working scripts per task — image_to_video, text_to_video, text_to_speech…

Mid-migration: new models land in deploy/, old ones haven't moved (Wan2.2).

source: vllm-omni repo · docs: stage_configs.md, adding_diffusion_model.md

III · SERVING & DEPLOYMENT

The repo, mapped — where things live

	DIRECTORY	ROLE	WHAT LIVES THERE
NEW	`vllm_omni/deploy/` YAML Templates	NEW SCHEMA	user-editable runtime YAMLs, auto-loaded — gpu mem, devices, connectors (migrated: Qwen3-Omni, TTS…)
OLD	`model_executor/stage_configs/`	LEGACY	old stage_args format, load via --stage-configs-path — Wan2.2's fp8 config still lives here
	`model_executor/models/`	AR / OMNI	Thinker/Talker/Code2wav implementations — OmniARScheduler, continuous batching
	`diffusion/models/`	DIT	pipeline_*.py + *_transformer.py per model — DiffusionEngine, step-wise sampling
	`recipes/ (external repo)`	DOCS	per-model setup guides & tuning recipes — community-editable, outside the codebase
	`examples/online_serving/`	RUNNABLE	working scripts per task — image_to_video, text_to_video, text_to_speech…

Mid-migration: new models land in deploy/, old ones haven't moved (Wan2.2).

source: vllm-omni repo · docs: stage_configs.md, adding_diffusion_model.md

III · SERVING & DEPLOYMENT

The repo, mapped — where things live

	DIRECTORY	ROLE	WHAT LIVES THERE
NEW	vllm_omni/deploy/ YAML Templates	NEW SCHEMA	user-editable runtime YAMLs, auto-loaded — gpu mem, devices, connectors (migrated: Qwen3-Omni, TTS…)
OLD	model_executor/stage_configs/	LEGACY	old stage_args format, load via --stage-configs-path — Wan2.2's fp8 config still lives here
Python	model_executor/models/	AR / OMNI	Thinker/Talker/Code2wav implementations — OmniARScheduler, continuous batching
Python	diffusion/models/	DIT	pipeline_*.py + *_transformer.py per model — DiffusionEngine, step-wise sampling
	recipes/ (external repo)	DOCS	per-model setup guides & tuning recipes — community-editable, outside the codebase
	examples/online_serving/	RUNNABLE	working scripts per task — image_to_video, text_to_video, text_to_speech…

Pipelines

Mid-migration: new models land in deploy/, old ones haven't moved (Wan2.2).

source: vllm-omni repo · docs: stage_configs.md, adding_diffusion_model.md

III · SERVING & DEPLOYMENT

The repo, mapped — where things live

	DIRECTORY	ROLE	WHAT LIVES THERE
NEW	vllm_omni/deploy/ YAML Templates	NEW SCHEMA	user-editable runtime YAMLs, auto-loaded — gpu mem, devices, connectors (migrated: Qwen3-Omni, TTS…)
OLD	model_executor/stage_configs/	LEGACY	old stage_args format, load via --stage-configs-path — Wan2.2's fp8 config still lives here
Python	model_executor/models/	AR / OMNI	Thinker/Talker/Code2wav implementations — OmniARScheduler, continuous batching
Python	diffusion/models/	DIT	pipeline_*.py + *_transformer.py per model — DiffusionEngine, step-wise sampling
Tutorials	recipes/ (external repo)	DOCS	per-model setup guides & tuning recipes — community-editable, outside the codebase
Tutorials	examples/online_serving/	RUNNABLE	working scripts per task — image_to_video, text_to_video, text_to_speech…

Pipelines

Mid-migration: new models land in deploy/, old ones haven't moved (Wan2.2).

source: vllm-omni repo · docs: stage_configs.md, adding_diffusion_model.md

III · SERVING & DEPLOYMENT

Diffusion, decoded — six terms you'll keep hearing

Reverse diffusion
learn to remove noise step by step — generation is denoising, run backwards

Timesteps
the countdown: how many denoise steps from pure noise to image (your latency dial)

Timestep embedding
tells the network "how noisy is this, where are we in the countdown"

The denoising loop
every step = a full forward pass through the DiT — that's why it's compute-bound

CFG
classifier-free guidance — two passes per step (with & without prompt), blended for adherence

Latent space (VAE)
diffusion's ZIP file — denoise in compressed space, VAE decodes to pixels at the end

VAE Patch Parallelism AT SCALE
split the VAE encode/decode across GPUs in patches — the TP trick for diffusion's heaviest non-DiT step · --vae-patch-parallel-size

HSDP AT SCALE
Hybrid Sharded Data Parallel — shard DiT weights within a node, replicate across nodes · --use-hsdp

full deep dive

FREE · the full diffusion explainer, illustrated — scan or hit **cloudthrill.ca/what-is-diffusion-explained**

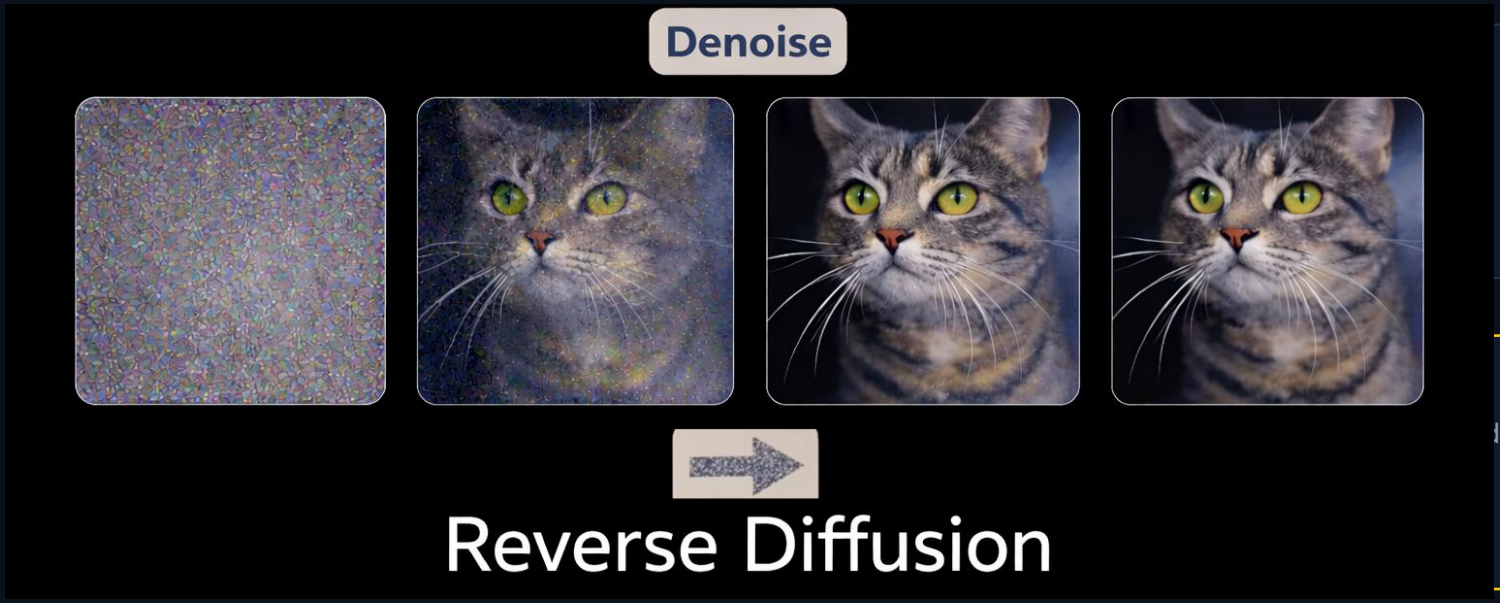
III · SERVING & DEPLOYMENT

Diffusion, decoded — six terms you'll keep hearing

Reverse diffusion
learn to remove noise step by step — generation is denoising, run backwards

Timesteps
the countdown: how many denoise steps from pure noise to image (your latency dial)

Timestep embedding
tells the network "how noisy is this, where are we in the countdown"



Latent space (VAE)
diffusion's ZIP file — denoise in compressed space, VAE decodes to pixels at the end

…l DiT weights within a node,

full deep dive

FREE · the full diffusion explainer, illustrated — scan or hit **cloudthrill.ca/what-is-diffusion-explained**

III · SERVING & DEPLOYMENT

Diffusion, decoded — six terms you'll keep hearing

Reverse diffusion
learn to remove noise step by step — generation is denoising, run backwards

Timesteps
the countdown: how many denoise steps from pure noise to image (your latency dial)

Timestep embedding
tells the network "how noisy is this, where are we in the countdown"

The denoising loop
every step = a full forward pass through the DiT — that's why it's compute-bound

CFG
classifier-free guidance — two passes per step (with & without prompt), blended for adherence

Latent space (VAE)
diffusion's ZIP file — denoise in compressed space, VAE decodes to pixels at the end

VAE Patch Parallelism AT SCALE
split the VAE encode/decode across GPUs in patches — the TP trick for diffusion's heaviest non-DiT step · --vae-patch-parallel-size

HSDP AT SCALE
Hybrid Sharded Data Parallel — shard DiT weights within a node, replicate across nodes · --use-hsdp

full deep dive

FREE · the full diffusion explainer, illustrated — scan or hit **cloudthrill.ca/what-is-diffusion-explained**

III · SERVING & DEPLOYMENT

Diffusion, decoded — six terms you'll keep hearing

Reverse diffusion
learn to remove noise step by step — generation is denoising, run backwards

Timesteps
the countdown: how many denoise steps from pure noise to image (your latency dial)

Timestep e
tells the networ
the countdown"

The denoising loop
every step = a full forward pass through the DiT — that's why it's compute-bound

CFG
classifier-free guidance — two passes per step (with & without prompt), blended for adherence

Latent spac
diffusion's ZIP f
VAE decodes t

What is Diffusion: from Noise → Pixels
`/what-is-diffusion-explained`

VAE Patch Parallelism AT SCALE
split the VAE encode/decode across GPUs in patches — the TP trick for diffusion's heaviest non-DiT step · --vae-patch-parallel-size

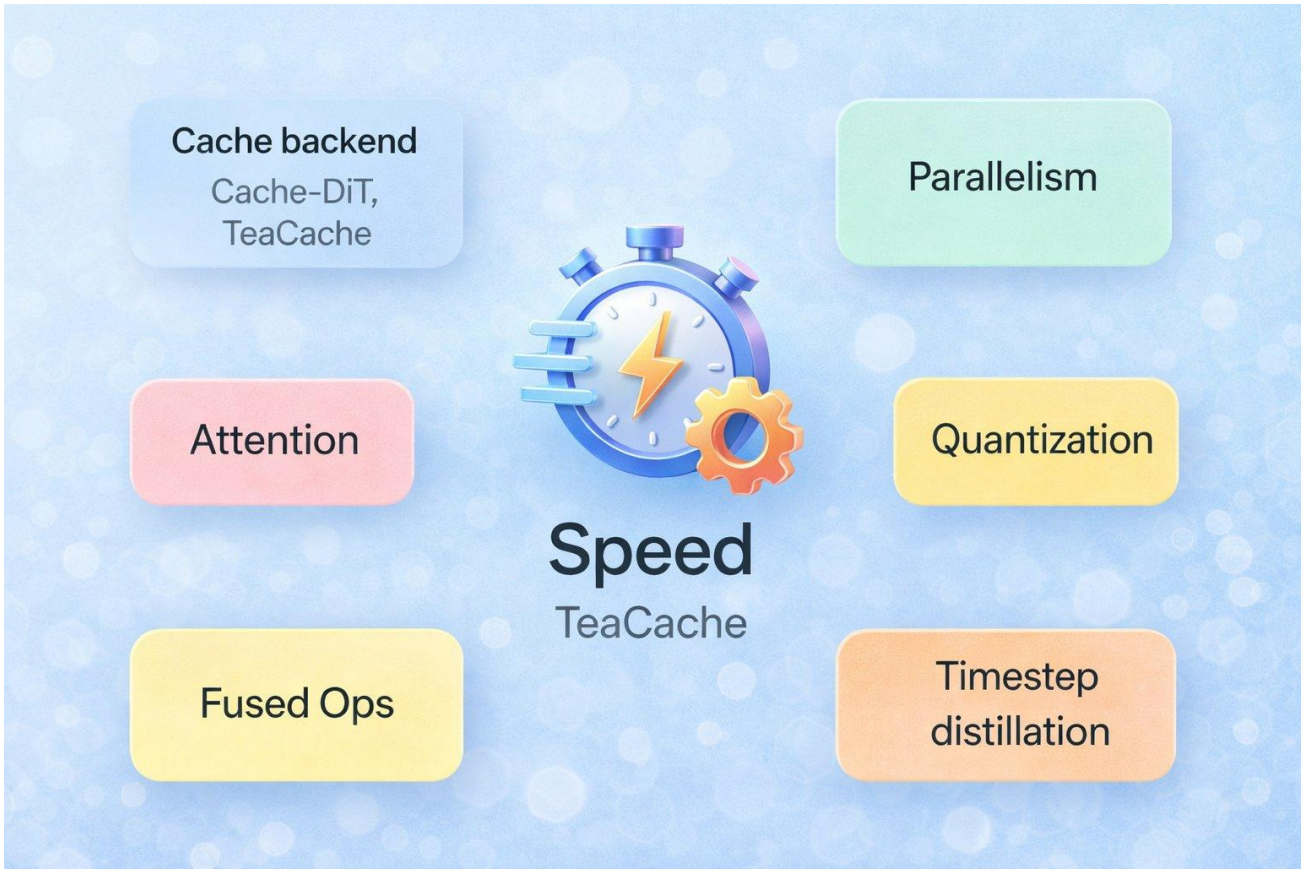
HSDP AT SCALE
Hybrid Sharded Data Parallel — shard DiT weights within a node, replicate across nodes · --use-hsdp

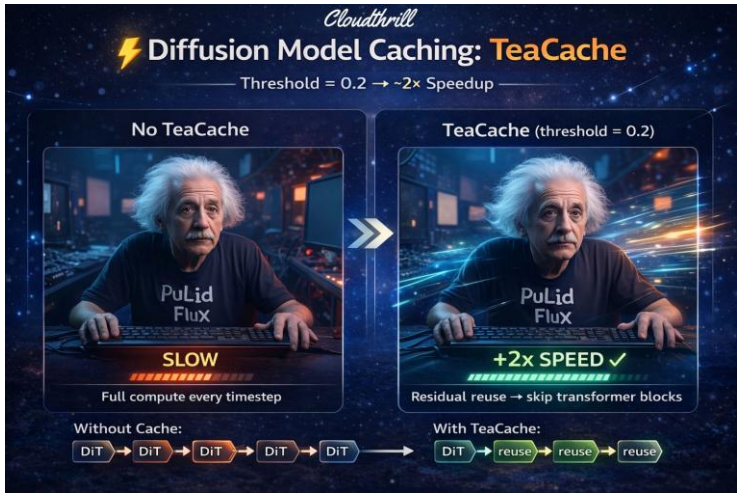
full deep dive

FREE · the full diffusion explainer, illustrated — scan or hit **cloudthrill.ca/what-is-diffusion-explained**

IV · OMNI ACCELERATION FEATURES

The acceleration toolbox





FREE

TeaCache, explained

threshold 0.2 → ~2× — the deep dive on this slide's cache lane

`cloudthrill.ca/what-is-teacache-explained`

Today: the **cache backend** lane — where our 4× lives.

IV · ACCELERATION

Supported features —

Lossy = a little quality for speed **Lossless** = math-identical, just more GPUs **Memory** = fit bigger on less

LOSSY · CACHING

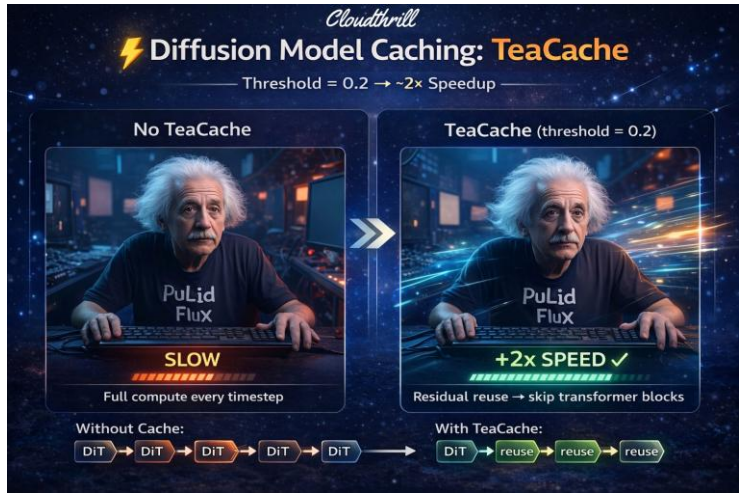
- **TeaCache**
  adaptive caching on modulated inputs — quick setup, ~2× on 1 GPU
- **Cache-DiT**
  DBCache / TaylorSeer / SCM — fine-grained, tunable quality↔speed

MEMORY · FIT BIGGER

- **CPU Offload**
  model components → CPU RAM — large models on consumer GPUs
- **Quantization**
  BF16 → FP8 / INT8 — minimal accuracy loss
- **VAE Patch Parallel**
  distributes VAE-decode tiling across GPUs

LOSSLESS · PARALLELISM

- **Ulysses-SP**
  seq parallel, all-to-all — hi-res / long video, 2–8 GPU
- **Ring-Attention**
  seq parallel, ring — long seqs, memory-tight, 2–8 GPU
- **CFG-Parallel**
  splits +/− CFG branches — CFG editing, 2 GPU
- **Tensor Parallel**
  shards layer weights — models too big for 1 GPU
- **Pipeline Parallel**
  block-wise DiT stages — lower per-GPU model memory
- **HSDP**
  FSDP2 weight shard — 14B+ on limited VRAM, stacks with SP
- **Expert Parallel**
  shards MoE experts — MoE diffusion (e.g. HunyuanImage3)



FREE

TeaCache, explained

threshold 0.2 → ~2× — the deep dive on this slide's cache lane

`cloudthrill.ca/what-is-teacache-explained`

source: vLLM-Omni docs · supported features · methods combine — see Feature Compatibility Table

IV · OMNI ACCELERATION FEATURES

Acceleration Support not one-size-fits-all · ImageGen

Model	TeaCache	Cache-DiT	SP	CFG-P	TP	PP	HSDP	CPU-Off	VAE-PP	Quant	Step
Qwen-Image-2512	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓
FLUX.2-dev	✓	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗
Stable-Diffusion3.5	✗	✓	✗	✓	✓	✗	✗	✓	◑	✗	✗
LongCat-Image	✓	✓	✓	✓	✓	✗	✗	✓	✗	✗	✗
HunyuanImage3	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✗
Z-Image	✓	✓	✓	?	✓	✗	✓	✗	◑	✓	✗
Bagel	✓	✓	✓	✓	✓	✗	✓	✓	✗	✗	✗
Cosmos3	✗	✓	✓	✓	✓	✗	✓	✓	◑	✓	✗

Across the board: Cache-DiT is the universal engine · TeaCache is hit-or-miss · parallelism is broadly available · ◑ = *partial*

*source: vLLM-Omni docs · **feature compatibility** · one representative per family*

IV · OMNI ACCELERATION FEATURES

Acceleration Support not one-size-fits-all · VideoGen

Model	TeaCache	Cache-DiT	SP	CFG-P	TP	PP	HSDP	CPU-Off	VAE-PP	Quant	Step
Wan2.2	✗	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗
Wan2.1-VACE	✗	✓	✓	✓	✓	✗	✓	✓	◑	✗	✗
LTX-2.3	✗	✓	✓	✓	✓	✗	✗	✓	◑	✗	✗
HunyuanVideo-1.5	✗	✓	✓	✓	✓	✗	✓	✓	✓	✓	✗
Helios	✗	✓	✓	✓	✓	✗	✓	✓	✗	✗	✗
DreamID-Omni	✗	✗	✗	✓	✗	✗	✓	✓	✗	✗	✗
Wan2.2-S2V	✗	✓	✗	✗	✓	✗	✗	✓	✓	✗	✗
Cosmos3	✗	✓	✓	✓	✓	✗	✓	✓	✓	✓	✗

TeaCache: a wall of ✗ on video. Wan2.2 needs per-model coefficients *and* a registered extractor — has neither. .

source: vLLM-Omni docs · feature compatibility · one representative per family · verify before recording

IV · OMNI ACCELERATION FEATURES

The acceleration dials, decoded

`--usp N` Ulysses-SP
image too big for one GPU? hand each GPU a slice of the pixels. N slices.

`--ring N` Ring-Attention
another way to slice the pixels — GPUs pass data in a ring. Stack with --usp.

`--cfg-parallel-size N` CFG-Parallel
CFG runs two passes (with prompt, without) — put them on two GPUs, not in sequence.

`--vae-patch-parallel-size N` VAE Patch Parallel
the final pixel-decode, split across GPUs in patches — for speed.

`--use-hsdp` Hybrid Sharded DP
model too big for one GPU? shard its weights within a node, copy across nodes.

`--tensor-parallel-size N` Tensor Parallel
split each layer's weights across N GPUs — classic big-model trick.

`--vae-use-tiling` VAE Tiling
decode the picture one tile at a time so a big frame fits in memory — for memory.

`--ulysses-mode` Ulysses mode
strict (default), or advanced_uaa when slice sizes don't divide evenly.

Two kinds of dials: split the **data** (pixels, sequences, passes) or split the **model** (weights, layers). Each one multiplies into your GPU count.

source: vllm-omni docs · sequence_parallel, parallelism_acceleration, cli/serve

Same 8 GPUs, two ways to split VideoGen (Wan)— the CFG fork

```
# distilled checkpoint — skip CFG
# all 8 GPUs go to sequence parallel
$ CUDA_VISIBLE_DEVICES=0-7 \
  vllm serve Wan-AI/Wan2.2-I2V-A14B-Diffusers \
    --omni --use-hsdp \
    --usp 8 \
    --vae-patch-parallel-size 8 \
    --vae-use-tiling
# no --cfg-parallel — distilled = 1 pass/step
```

```
# standard checkpoint — CFG on
# 2 × CFG branches × 4-way sequence parallel
$ CUDA_VISIBLE_DEVICES=0-7 \
  vllm serve Wan-AI/Wan2.2-I2V-A14B-Diffusers \
    --omni --use-hsdp \
    --cfg-parallel-size 2 --usp 4 \
    --vae-patch-parallel-size 8 \
    --vae-use-tiling
# 2 × 4 = same 8 GPUs, different math
```

```
# the same dials, smaller rigs — Qwen-Image:
$ vllm serve Qwen/Qwen-Image --omni --tensor-parallel-size 2 --vae-patch-parallel-size 2   # 2 GPUs
$ vllm serve Qwen/Qwen-Image --omni --usp 2 --ring 2                                       # 4 GPUs, Ulysses + Ring
```

CFG costs a ×2 every step. Distilled models drop it — and hand the budget back to parallelism.

source: recipes/Wan-AI/Wan2.2-I2V.md · vllm-omni docs

IV · OMNI ACCELERATION FEATURES

Going Multinode (Qwen-omni)— the overlay

```yaml
# my_qwen3_omni_multinode.yaml
base_config: .../deploy/qwen3_omni_moe.yaml

connectors:
  mooncake_connector:
    name: MooncakeStoreConnector
    extra:
      metadata_server: http://HOST:8080/...
      master: MASTER_HOST:50051
      segment: 512000000   # 512 MB
      proto: tcp

stages:
  - stage_id: 0
    output_connectors: {to_stage_1: mooncake_connector}
  - stage_id: 1
    input_connectors:  {from_stage_0: mooncake_connector}
    output_connectors: {to_stage_2: mooncake_connector}
  - stage_id: 2
    input_connectors:  {from_stage_1: mooncake_connector}
```

Overlay, don’t rewrite

inherit the shipped config via base_config — override only the wiring

Mooncake = the transfer engine

remember the disagg slide? this is that memory-pool path, configured

Stages wired explicitly

each hop declares its connector — stage 0 → 1 → 2 across machines

Same model, zero code

from one H100 to a multinode mesh — it’s all deploy YAML

source: vllm-omni docs · deploy overlay pattern · MooncakeStoreConnector

DEMO

DAY 0 Deploy NVIDIA Cosmos 3 on vLLM-Omni

deploy-cosmos3.sh

```
docker run --runtime nvidia --gpus all \
  -v ~/.cache/huggingface:/root/.cache/huggingface \
  -v "$(pwd):/workspace" \
  -p 8000:8000 \
  --ipc=host \
  vllm/vllm-omni:cosmos3 \
  vllm serve nvidia/Cosmos3-Nano \
  --omni \
  --model-class-name Cosmos3OmniDiffusersPipeline \
  --allowed-local-media-path /
```

One **OpenAI-compatible** server · text→image · text→video · image→video · +sound · +action

vLLM-Omni × Cosmos 3

DEMO — SERVE COMMANDS

live · Nebius H100 · vllm-omni v0.22.0

kosseila@nebius-h100: ~/vllm-omni-demo

```
# IMAGE · diffusion (DiT)
$ vllm serve Tongyi-MAI/Z-Image-Turbo --omni

# VIDEO · diffusion + Cache-DiT
$ vllm serve Wan-AI/Wan2.2-TI2V-5B-Diffusers --omni --cache-backend cache_dit

# SPEECH · AR talker + codec
$ vllm serve Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice --omni

# WORLD MODEL · Cosmos 3 (NVIDIA)
$ vllm serve nvidia/Cosmos3-Nano --omni --model-class-name Cosmos3OmniDiffusersPipeline
```

Same standalone commands `vllm serve … --omni` — only the model changes. **Two brains, one command.**

DEMO — API REQUESTS, EVERY MODALITY

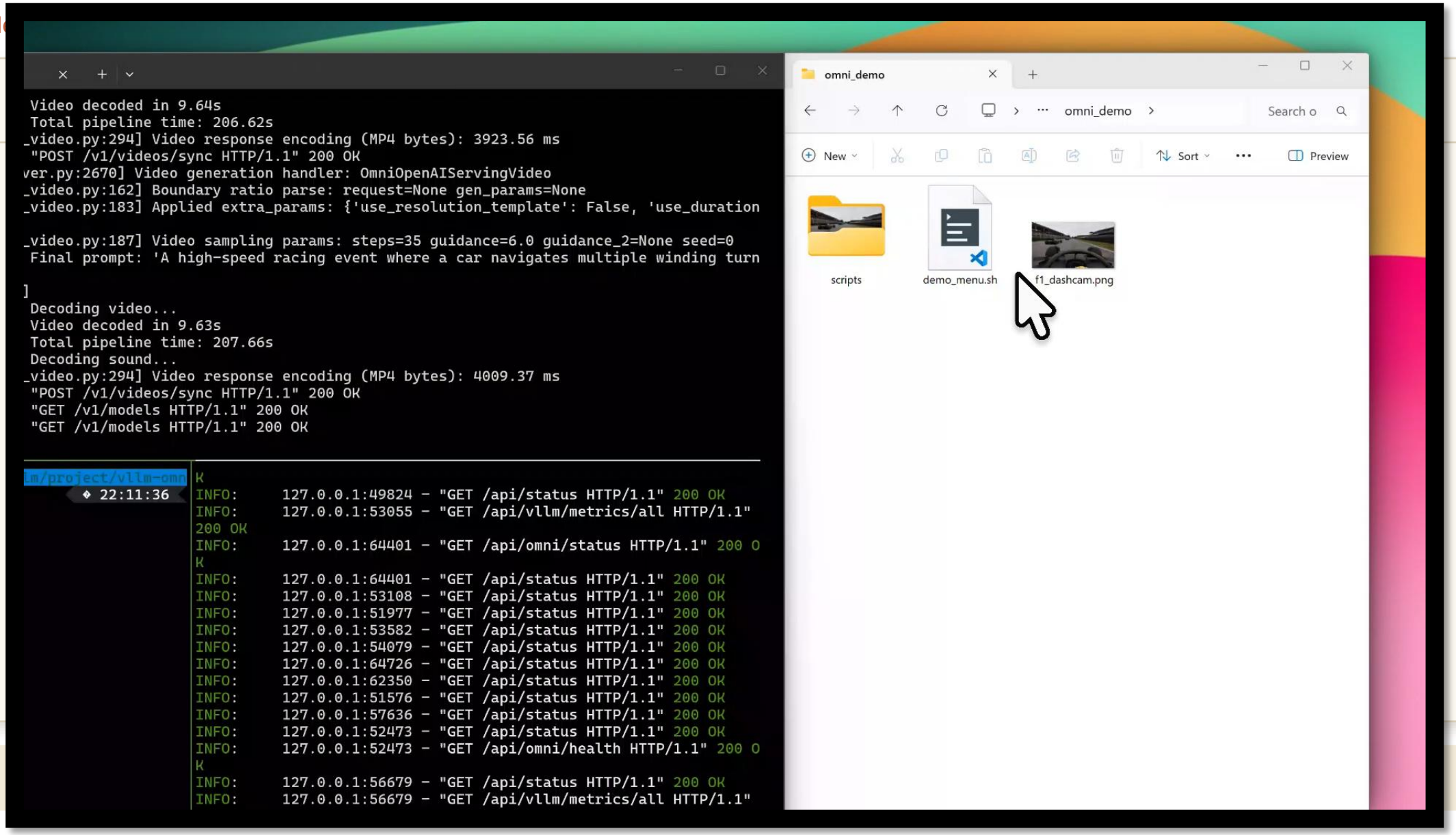
Nebius H100 node | API Calls

kosseila@nebius-h100: ~/vllm-omni-demo | demo_menu.sh

```
# IMAGE (Z-Image-Turbo) or vLLM Playground →  whale.png
$ curl …/v1/images/generations -d '{prompt:"whale…", seed:100}'
  → 200 OK · b64_json → whale.png · 1024×1024 · ~10s

# IMAGE → VIDEO (Wan2.2) with CACHE-Dit ·  feed the same whale.png
$ curl …/v1/videos -F input_reference=@whale.png -F prompt=…
  → {"id":"vid_…","status":"queued"}   ▸ poll ▸ whale_i2v.mp4

# SPEECH (Qwen3-TTS) →  tts_demo.wav   (AR talker + codec)
$ curl …/v1/audio/speech -d '{input:"Sight. Sound. Voice…", voice:"vivian"}'
  → 24 kHz wav · ~3s to first audio

# Cosmos3-Nano · :8000 · text → image → video (F1 dashcam)
$ curl …/v1/videos/sync -F input_reference=@f1_dashcam.png -F prompt=…turns
  → cosmos3_f1_i2v.mp4 · 1280×720
```

Using Custom Demo_Menu: one script, every modality, same OpenAI-style API.

DEMO — API REQUESTS, EVERY MODALITY



DEMO — API REQUESTS, EVERY MODALITY

Nebiu

Cache-DiT on Wan2.2 TI2V-5B

Same clip, same seed — 81 frames (~3s), 720p, 50 steps

Baseline

108s — 1:48

Cache-DiT

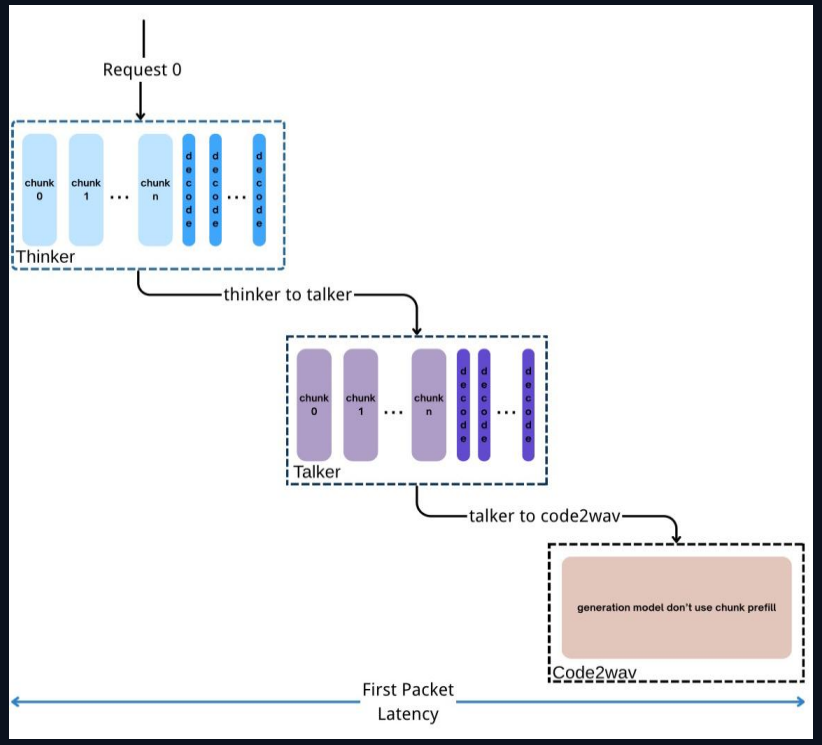
27s — 0:27

4× faster

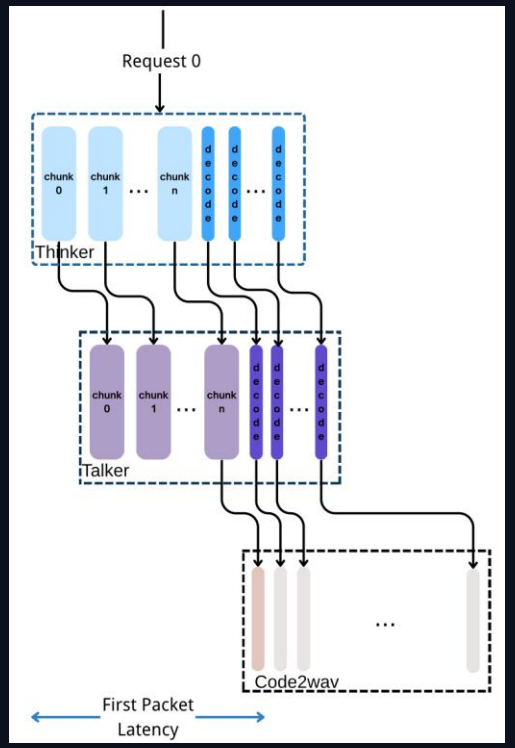
~75% less wall-clock, same output

Using Custom Demo_Menu: one script, every modality, same OpenAI-style API.

Async Chunk & Audio Streaming Output



E2E generation



Streaming generation

- **Pipeline Between Stages**: Asynchronous chunked computation and communication across stages
- **Audio Streaming Output:** The waveform is output immediately after Talker generates each token.
- **APIServer Support:** OpenAI v1/chat/completions with "stream" argument

Streaming audio: don’t wait for the whole answer

Async Chunk

12.4×

faster to first audio packet

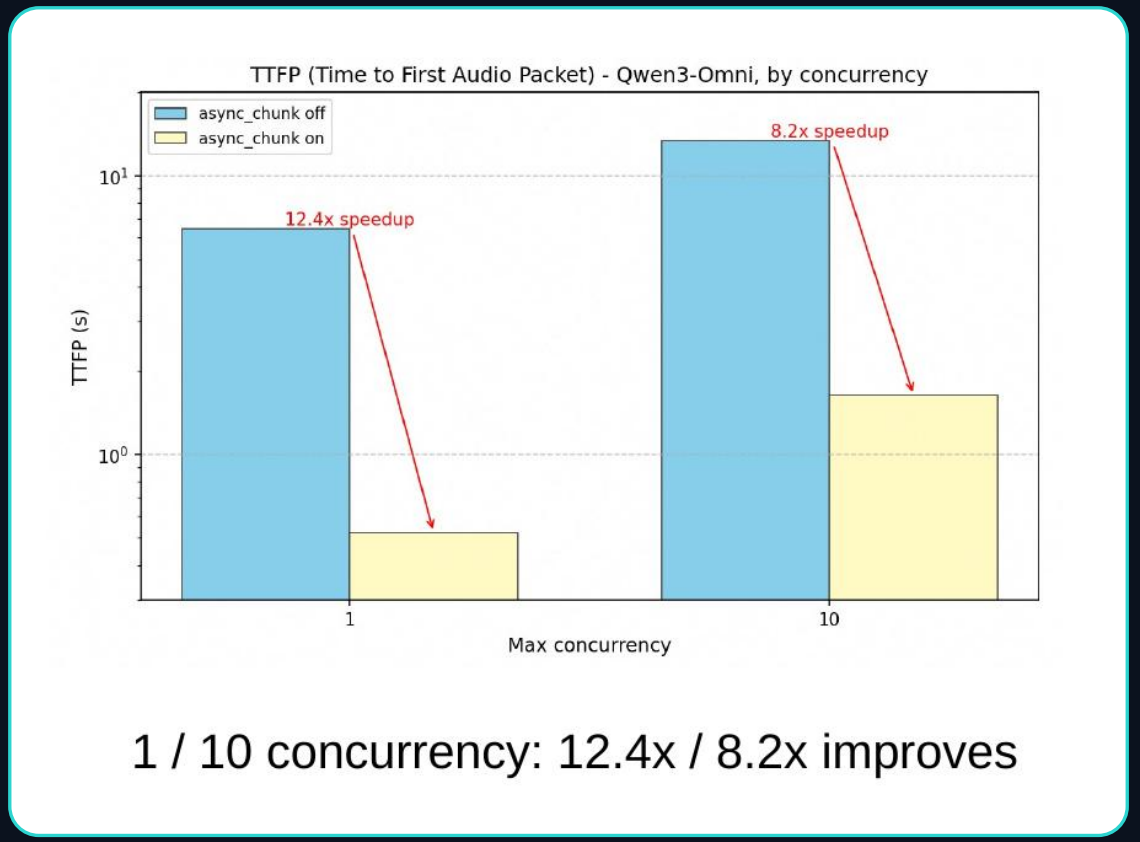
single request · Qwen3-Omni

8.2×

at 10× concurrency

How: stages pipeline **chunks** of audio asynchronously — Talker streams while Code2wav decodes.

- Qwen3-Omni



- RTF= Real Time Capable

source: vLLM-Omni team · vLLM Office Hours #46 / project docs

Streaming audio: don’t wait for the whole answer

Async Chunk

12.4×

faster to first audio packet

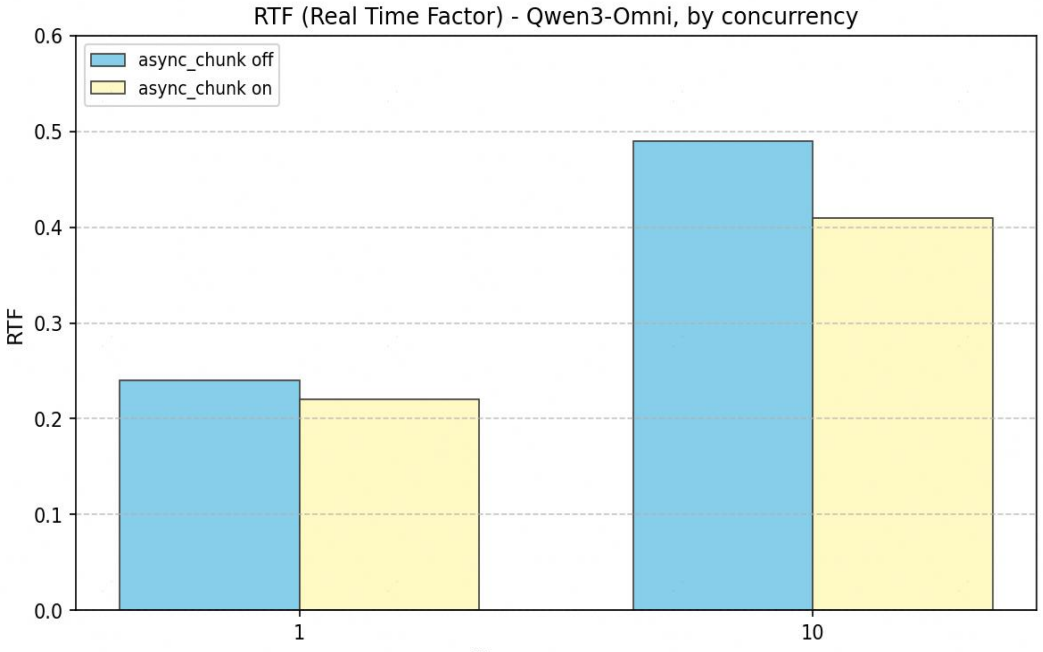
single request · Qwen3-Omni

8.2×

at 10× concurrency

How: stages pipeline **chunks** of audio asynchronously — Talker streams while Code2wav decodes.

- Qwen3-Omni



1 / 10 concurrency: 1.1 x / 1.2 x improves

- RTF= Real Time Capable

source: vLLM-Omni team · vLLM Office Hours #46 / project docs

Streaming audio: don’t wait for the whole answer

Async Chunk

12.4×

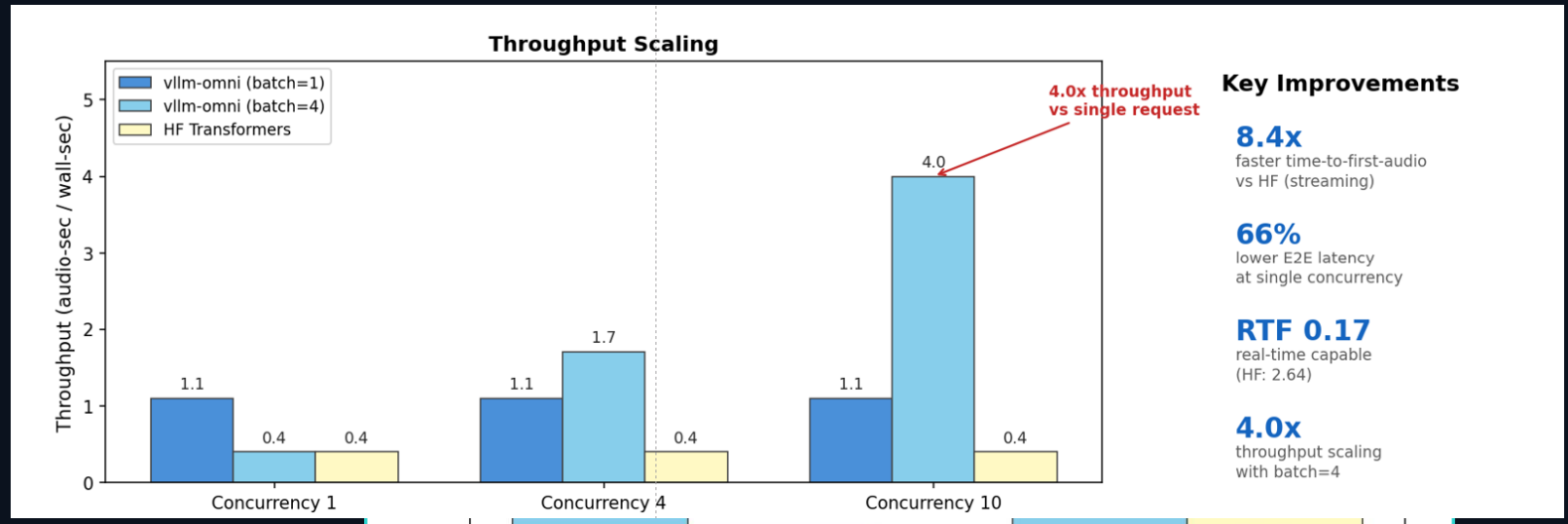
faster to first audio packet

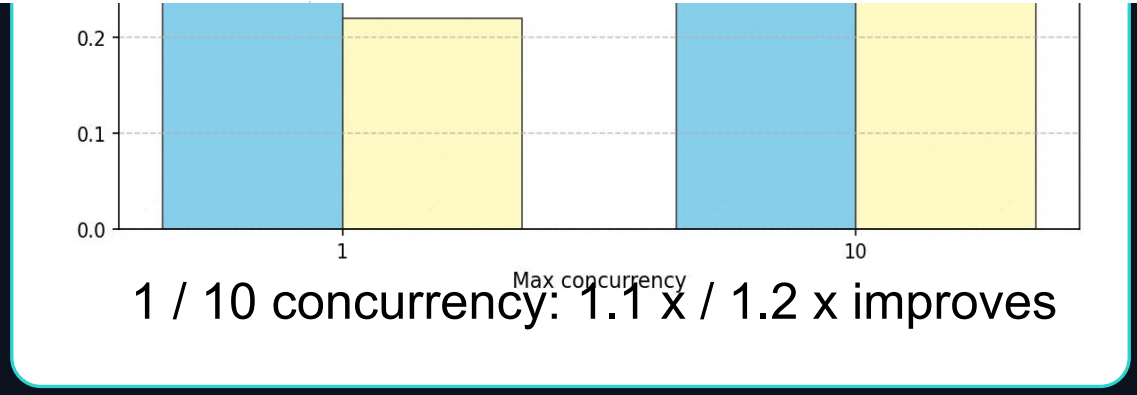
single request · Qwen3-Omni

8.2×

at 10× concurrency

How: stages pipeline **chunks** of audio asynchronously — Talker streams while Code2wav decodes.





1 / 10 concurrency: 1.1 x / 1.2 x improves

- RTF= Real Time Capable

source: vLLM-Omni team · vLLM Office Hours #46 / project docs

VI · v0.22 & BEYOND

What's Just shipped — and Roadmap

v0.22.0 · JUST SHIPPED

- ✓ **Cosmos 3 world models**
  day-0 with NVIDIA — text/img/audio/video/action
- ✓ **Robot serving**
  DreamZero + OpenPI realtime API — embodied models
- ✓ **Production TTS**
  Qwen3-TTS, Qwen3-Omni, VoxCPM2 — streaming speech
- ✓ **Faster diffusion**
  Wan 2.2, HunyuanVideo 1.5, LTX-2.3 — cache + parallel
- ✓ **Broader quantization**
  FP8 / INT8, MXFP4 / MXFP8, W4A16, ModelOpt

ON THE ROADMAP

- → **Day-0 models + wider hardware**
  more backends
- → **Diffusers backend**
  broader model reach
- → **RL post-training**
  veRL & veOmni
- → **Video streaming output**
  audio streams today
- → **Full disaggregation at scale**
  OmniConnector & OmniRouter
- → **Engine upgrades**
  ModelRunner V2 + diffusion continuous batching

source: vllm-omni v0.22.0 release notes · @vllm_project

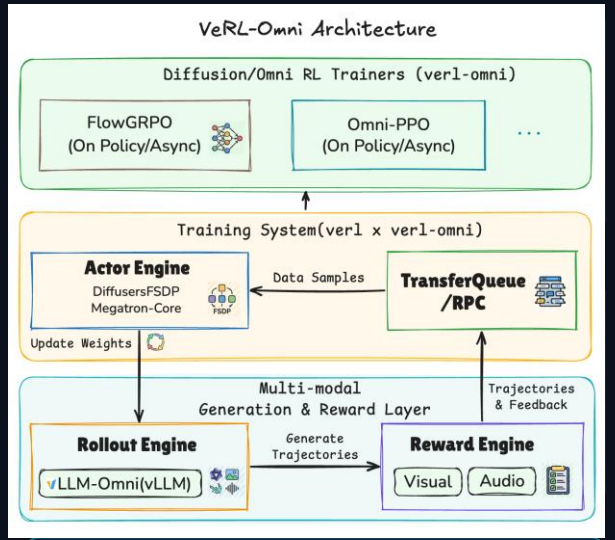
VI · v0.22 & BEYOND

What's Just shipped — and Roadmap

v0.22.0 · JUST SHIPPED

- **Cosmos 3 world models**
  day-0 with NVIDIA — text/img/audio/video/action
- **Robot serving**
  DreamZero + OpenPI realtime API — embodied models
- **Production TTS**
  Qwen3-TTS, Qwen3-Omni, VoxCPM2 — streaming speech
- **Faster diffusion**
  Wan 2.2, HunyuanVideo 1.5, LTX-2.3 — cache + parallel
- **Broader quantization**
  FP8 / INT8, MXFP4 / MXFP8, W4A16, ModelOpt

ON THE ROADMAP



- **Full disaggregation at scale**
  OmniConnector & OmniRouter
- **Diffusers backend**
  broader model reach
- **Video streaming output**
  audio streams today
- **Engine upgrades**
  ModelRunner V2 + diffusion continuous batching

source: vllm-omni v0.22.0 release notes · @vllm_project

VI · v0.22 & BEYOND

What's Just shipped — and Roadmap

v0.22.0 · JUST SHIPPED

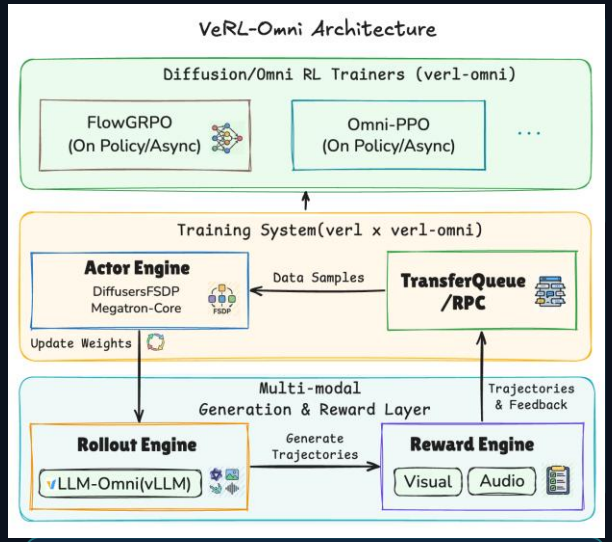
Day-0 partners: NVIDIA · and a fast-growing OSS community.

Jump in — it's all open source.

vLLM-Omni

source: vllm-omni v0.22.0 release notes · @vllm_p

ON THE ROADMAP



Full disaggregation at scale
OmniConnector & OmniRouter

Diffusers backend
broader model reach

Video streaming output
audio streams today

Engine upgrades
ModelRunner V2 + diffusion
continuous batching

VI · v0.22 & BEYOND

What's Just shipped — and Roadmap

v0.22.0 · JUST SHIPPED

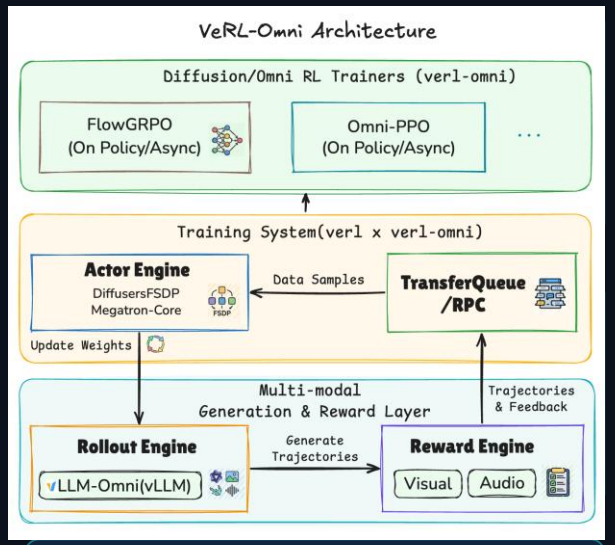
Day-0 partners: NVIDIA · and a fast-growing OSS community.

Jump in — it's all open source.

vLLM-Omni

source: vllm-omni v0.22.0 release notes · @vllm_p

ON THE ROADMAP



Full disaggregation at scale
OmniConnector & OmniRouter

Diffusers backend
broader model reach

Video streaming output
audio streams today

Engine upgrades
ModelRunner V2 + diffusion continuous batching

https://communityinviter.com/apps/vllm-dev/join-vllm-developers-slack

GO DEEPER

More Deep dives below. Check out Cloudthrill.ca/blog.

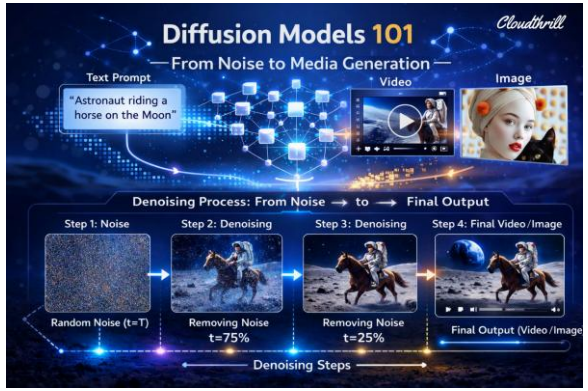
Beginners 1: The Fundamentals
`/what-is-vllm`

Beginners 2: Key Features
`/what-is-vllm-features`

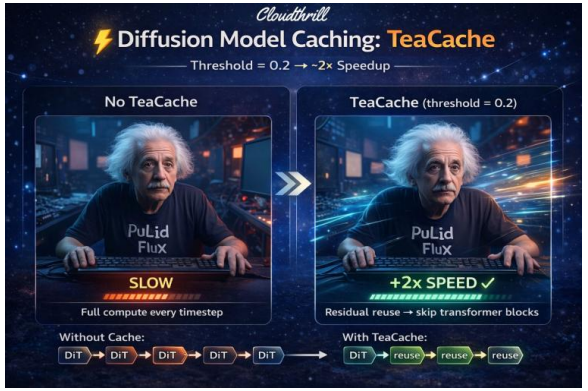
Beginners 3: Deployment
`/vllm-deloyment`

KV-Cache, like I'm 5
`/kv_cache-explained`

What is vLLM-Omni?
`/what-is-vllm-omni`



What is Diffusion: from Noise → Pixels
`/what-is-diffusion-explained`



TeaCache, explained
`/what-is-teacache-explained`

all of it
`cloudthrill.ca/blog`